projectmanager Test Documentation

0.0.1

Generated by Doxygen 1.16.1

1 Directory Hierarchy 1
1.1 Directories . . . 1

2 Namespace Index 5
2.1 Package List . . . 5

3 Class Index 7
3.1 Class List . . . 7

4 File Index 9
4.1 File List . . . 9

5 Directory Documentation 11
5.1 projectmanager-app/src/test/java/com/projectmanagement/app Directory Reference . . . 11
5.2 projectmanager-app/src/test/java/com Directory Reference . . . 11
5.3 projectmanager-app/src/test/java Directory Reference . . . 11
5.4 projectmanager-app/src/test/java/com/projectmanagement/lib Directory Reference . . . 11
5.5 projectmanager-app/src/test/java/com/projectmanagement/lib/models Directory Reference . . . 12
5.6 projectmanager-app/src/test/java/com/projectmanagement Directory Reference . . . 12
5.7 projectmanager-app Directory Reference . . . 12
5.8 projectmanager-app/src/test/java/com/projectmanagement/lib/services Directory Reference . . . 12
5.9 projectmanager-app/src Directory Reference . . . 12
5.10 projectmanager-app/src/test/java/com/projectmanagement/lib/storage Directory Reference . . . 13
5.11 projectmanager-app/src/test Directory Reference . . . 13

6 Namespace Documentation 15
6.1 Package com.projectmanagement.app . . . 15
6.2 Package com.projectmanagement.lib.models . . . 15
6.3 Package com.projectmanagement.lib.services . . . 15
6.4 Package com.projectmanagement.lib.storage . . . 15

7 Class Documentation 17
7.1 com.projectmanagement.app.MainTest Class Reference . . . 17
7.1.1 Detailed Description . . . 17
7.1.2 Member Function Documentation . . . 17
7.1.2.1 restoreSystemInputOutput() . . . 17
7.1.2.2 setUpOutput() . . . 17
7.1.2.3 testMain_FullNavigation() . . . 18
7.1.2.4 testMain_GuestModeAndExit() . . . 19
7.2 com.projectmanagement.lib.services.ProjectServiceTest Class Reference . . . 20
7.2.1 Detailed Description . . . 20
7.2.2 Member Function Documentation . . . 20
7.2.2.1 setup() . . . 20
7.2.2.2 testAddTaskToProject_InvalidInputs_ThrowsException() . . . 20

7.2.2.3 testAddTaskToProject_Success() . . . 21
7.2.2.4 testCreateProject_NullInputs_ThrowsException() . . . 21
7.2.2.5 testCreateProject_Success() . . . 21
7.2.2.6 testGetAllProjects() . . . 21
7.3 com.projectmanagement.lib.models.ProjectTest Class Reference . . . 22
7.3.1 Detailed Description . . . 22
7.3.2 Member Function Documentation . . . 22
7.3.2.1 setUp() . . . 22
7.3.2.2 tearDown() . . . 22
7.3.2.3 testAddRemoveTask() . . . 22
7.3.2.4 testDefaultConstructor() . . . 23
7.3.2.5 testDisplayDetails() . . . 23
7.3.2.6 testParameterizedConstructor() . . . 23
7.3.2.7 testSettersAndGetters() . . . 23
7.4 com.projectmanagement.lib.services.ReportServiceTest Class Reference . . . 24
7.4.1 Detailed Description . . . 24
7.4.2 Member Function Documentation . . . 24
7.4.2.1 setUp() . . . 24
7.4.2.2 testGenerateGeneralSystemReport() . . . 24
7.4.2.3 testGenerateGeneralSystemReport_Empty() . . . 25
7.4.2.4 testGenerateGeneralSystemReport_EmptyLists() . . . 25
7.4.2.5 testGenerateTaskStatusReport() . . . 25
7.4.2.6 testGenerateTaskStatusReport_Empty() . . . 25
7.5 com.projectmanagement.lib.storage.RepositoryFactoryTest Class Reference . . . 26
7.5.1 Detailed Description . . . 26
7.5.2 Member Function Documentation . . . 26
7.5.2.1 testGetRepository_BinaryStorage() . . . 26
7.5.2.2 testGetRepository_MySQLStorage() . . . 26
7.5.2.3 testGetRepository_SQLiteStorage() . . . 27
7.5.2.4 testGetRepository_UnsupportedStorageType() . . . 27
7.6 com.projectmanagement.lib.services.TaskServiceTest Class Reference . . . 27
7.6.1 Detailed Description . . . 28
7.6.2 Member Function Documentation . . . 28
7.6.2.1 setup() . . . 28
7.6.2.2 testAssignTaskToUser_InvalidInputs_ThrowsException() . . . 28
7.6.2.3 testAssignTaskToUser_Success() . . . 28
7.6.2.4 testGetAllTasks() . . . 29
7.6.2.5 testRegisterTask_NullFields_ThrowsException() . . . 29
7.6.2.6 testRegisterTask_Success() . . . 29
7.6.2.7 testUpdateTaskStatus_InvalidInputs_ThrowsException() . . . 29
7.6.2.8 testUpdateTaskStatus_Success() . . . 30
7.7 com.projectmanagement.lib.models.TaskTest Class Reference . . . 30

7.7.1 Detailed Description . . . 30
7.7.2 Member Function Documentation . . . 30
7.7.2.1 setUp() . . . 30
7.7.2.2 tearDown() . . . 30
7.7.2.3 testDefaultConstructor() . . . 31
7.7.2.4 testDisplayDetailsAssigned() . . . 31
7.7.2.5 testDisplayDetailsUnassigned() . . . 31
7.7.2.6 testParameterizedConstructor() . . . 31
7.7.2.7 testSettersAndGetters() . . . 32
7.8 com.projectmanagement.lib.services.UserServiceTest Class Reference . . . 32
7.8.1 Detailed Description . . . 32
7.8.2 Member Function Documentation . . . 32
7.8.2.1 setup() . . . 32
7.8.2.2 testGetAllUsers() . . . 33
7.8.2.3 testLogin_InvalidCredentials() . . . 33
7.8.2.4 testLogin_NoUsersInDb() . . . 33
7.8.2.5 testLogin_NullOrEmptyInputs() . . . 33
7.8.2.6 testLogin_Success() . . . 34
7.8.2.7 testRegisterUser_NullFields_ThrowsException() . . . 34
7.8.2.8 testRegisterUser_Success() . . . 34
7.9 com.projectmanagement.lib.models.UserTest Class Reference . . . 35
7.9.1 Detailed Description . . . 35
7.9.2 Member Function Documentation . . . 35
7.9.2.1 setUp() . . . 35
7.9.2.2 tearDown() . . . 35
7.9.2.3 testDefaultConstructor() . . . 36
7.9.2.4 testDisplayDetails() . . . 36
7.9.2.5 testParameterizedConstructor() . . . 36
7.9.2.6 testSetters() . . . 36

8 File Documentation **37**
8.1 projectmanager-app/src/test/java/com/projectmanagement/app/MainTest.java File Reference . . . 37
8.2 MainTest.java . . . 37
8.3 projectmanager-app/src/test/java/com/projectmanagement/lib/models/ProjectTest.java File Reference 40
8.4 ProjectTest.java . . . 40
8.5 projectmanager-app/src/test/java/com/projectmanagement/lib/models/TaskTest.java File Reference . 42
8.6 TaskTest.java . . . 42
8.7 projectmanager-app/src/test/java/com/projectmanagement/lib/models/UserTest.java File Reference . 43
8.8 UserTest.java . . . 44
8.9 projectmanager-app/src/test/java/com/projectmanagement/lib/services/ProjectServiceTest.java File Reference . . . 45
8.10 ProjectServiceTest.java . . . 45

8.11 projectmanager-app/src/test/java/com/projectmanagement/lib/services/ReportServiceTest.java File Reference . . . 46
8.12 ReportServiceTest.java . . . 47
8.13 projectmanager-app/src/test/java/com/projectmanagement/lib/services/TaskServiceTest.java File Reference . . . 48
8.14 TaskServiceTest.java . . . 48
8.15 projectmanager-app/src/test/java/com/projectmanagement/lib/services/UserServiceTest.java File Reference . . . 50
8.16 UserServiceTest.java . . . 51
8.17 projectmanager-app/src/test/java/com/projectmanagement/lib/storage/BinaryRepositoryTest.java File Reference . . . 52
8.18 BinaryRepositoryTest.java . . . 53
8.19 projectmanager-app/src/test/java/com/projectmanagement/lib/storage/MySQLRepositoryTest.java File Reference . . . 54
8.20 MySQLRepositoryTest.java . . . 55
8.21 projectmanager-app/src/test/java/com/projectmanagement/lib/storage/RepositoryFactoryTest.java File Reference . . . 57
8.22 RepositoryFactoryTest.java . . . 57
8.23 projectmanager-app/src/test/java/com/projectmanagement/lib/storage/SQLiteRepositoryTest.java File Reference . . . 58
8.24 SQLiteRepositoryTest.java . . . 59

Chapter 1

Directory Hierarchy

1.1 Directories

app . . . 11
MainTest.java . . . 37
com . . . 11
projectmanagement . . . 12
app . . . 11
MainTest.java . . . 37
lib . . . 11
models . . . 12
ProjectTest.java . . . 40
TaskTest.java . . . 42
UserTest.java . . . 43
services . . . 12
ProjectServiceTest.java . . . 45
ReportServiceTest.java . . . 46
TaskServiceTest.java . . . 48
UserServiceTest.java . . . 50
storage . . . 13
BinaryRepositoryTest.java . . . 52
MySQLRepositoryTest.java . . . 54
RepositoryFactoryTest.java . . . 57
SQLiteRepositoryTest.java . . . 58
java . . . 11
com . . . 11
projectmanagement . . . 12
app . . . 11
MainTest.java . . . 37
lib . . . 11
models . . . 12
ProjectTest.java . . . 40
TaskTest.java . . . 42
UserTest.java . . . 43
services . . . 12
ProjectServiceTest.java . . . 45
ReportServiceTest.java . . . 46
TaskServiceTest.java . . . 48
UserServiceTest.java . . . 50

storage . . . 13
BinaryRepositoryTest.java . . . 52
MySQLRepositoryTest.java . . . 54
RepositoryFactoryTest.java . . . 57
SQLiteRepositoryTest.java . . . 58
lib . . . 11
models . . . 12
ProjectTest.java . . . 40
TaskTest.java . . . 42
UserTest.java . . . 43
services . . . 12
ProjectServiceTest.java . . . 45
ReportServiceTest.java . . . 46
TaskServiceTest.java . . . 48
UserServiceTest.java . . . 50
storage . . . 13
BinaryRepositoryTest.java . . . 52
MySQLRepositoryTest.java . . . 54
RepositoryFactoryTest.java . . . 57
SQLiteRepositoryTest.java . . . 58
models . . . 12
ProjectTest.java . . . 40
TaskTest.java . . . 42
UserTest.java . . . 43
projectmanagement . . . 12
app . . . 11
MainTest.java . . . 37
lib . . . 11
models . . . 12
ProjectTest.java . . . 40
TaskTest.java . . . 42
UserTest.java . . . 43
services . . . 12
ProjectServiceTest.java . . . 45
ReportServiceTest.java . . . 46
TaskServiceTest.java . . . 48
UserServiceTest.java . . . 50
storage . . . 13
BinaryRepositoryTest.java . . . 52
MySQLRepositoryTest.java . . . 54
RepositoryFactoryTest.java . . . 57
SQLiteRepositoryTest.java . . . 58
projectmanager-app . . . 12
src . . . 12
test . . . 13
java . . . 11
com . . . 11
projectmanagement . . . 12
app . . . 11
MainTest.java . . . 37
lib . . . 11
models . . . 12
ProjectTest.java . . . 40
TaskTest.java . . . 42
UserTest.java . . . 43
services . . . 12

ProjectServiceTest.java . . . 45
ReportServiceTest.java . . . 46
TaskServiceTest.java . . . 48
UserServiceTest.java . . . 50
storage . . . 13
BinaryRepositoryTest.java . . . 52
MySQLRepositoryTest.java . . . 54
RepositoryFactoryTest.java . . . 57
SQLiteRepositoryTest.java . . . 58
services . . . 12
ProjectServiceTest.java . . . 45
ReportServiceTest.java . . . 46
TaskServiceTest.java . . . 48
UserServiceTest.java . . . 50
src . . . 12
test . . . 13
java . . . 11
com . . . 11
projectmanagement . . . 12
app . . . 11
MainTest.java . . . 37
lib . . . 11
models . . . 12
ProjectTest.java . . . 40
TaskTest.java . . . 42
UserTest.java . . . 43
services . . . 12
ProjectServiceTest.java . . . 45
ReportServiceTest.java . . . 46
TaskServiceTest.java . . . 48
UserServiceTest.java . . . 50
storage . . . 13
BinaryRepositoryTest.java . . . 52
MySQLRepositoryTest.java . . . 54
RepositoryFactoryTest.java . . . 57
SQLiteRepositoryTest.java . . . 58
storage . . . 13
BinaryRepositoryTest.java . . . 52
MySQLRepositoryTest.java . . . 54
RepositoryFactoryTest.java . . . 57
SQLiteRepositoryTest.java . . . 58
test . . . 13
java . . . 11
com . . . 11
projectmanagement . . . 12
app . . . 11
MainTest.java . . . 37
lib . . . 11
models . . . 12
ProjectTest.java . . . 40
TaskTest.java . . . 42
UserTest.java . . . 43
services . . . 12
ProjectServiceTest.java . . . 45
ReportServiceTest.java . . . 46
TaskServiceTest.java . . . 48
UserServiceTest.java . . . 50

storage 13
BinaryRepositoryTest.java 52
MySQLRepositoryTest.java 54
RepositoryFactoryTest.java 57
SQLiteRepositoryTest.java 58

Chapter 2

Namespace Index

2.1 Package List

Here are the packages with brief descriptions (if available):

com.projectmanagement.app 15
com.projectmanagement.lib.models 15
com.projectmanagement.lib.services 15
com.projectmanagement.lib.storage 15

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

com.projectmanagement.app.MainTest 17
com.projectmanagement.lib.services.ProjectServiceTest 20
com.projectmanagement.lib.models.ProjectTest 22
com.projectmanagement.lib.services.ReportServiceTest 24
com.projectmanagement.lib.storage.RepositoryFactoryTest 26
com.projectmanagement.lib.services.TaskServiceTest 27
com.projectmanagement.lib.models.TaskTest 30
com.projectmanagement.lib.services.UserServiceTest 32
com.projectmanagement.lib.models.UserTest 35

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

projectmanager-app/src/test/java/com/projectmanagement/app/MainTest.java 37
projectmanager-app/src/test/java/com/projectmanagement/lib/models/ProjectTest.java 40
projectmanager-app/src/test/java/com/projectmanagement/lib/models/TaskTest.java 42
projectmanager-app/src/test/java/com/projectmanagement/lib/models/UserTest.java 43
projectmanager-app/src/test/java/com/projectmanagement/lib/services/ProjectServiceTest.java 45
projectmanager-app/src/test/java/com/projectmanagement/lib/services/ReportServiceTest.java 46
projectmanager-app/src/test/java/com/projectmanagement/lib/services/TaskServiceTest.java 48
projectmanager-app/src/test/java/com/projectmanagement/lib/services/UserServiceTest.java 50
projectmanager-app/src/test/java/com/projectmanagement/lib/storage/BinaryRepositoryTest.java 52
projectmanager-app/src/test/java/com/projectmanagement/lib/storage/MySQLRepositoryTest.java 54
projectmanager-app/src/test/java/com/projectmanagement/lib/storage/RepositoryFactoryTest.java 57
projectmanager-app/src/test/java/com/projectmanagement/lib/storage/SQLiteRepositoryTest.java 58

Chapter 5

Directory Documentation

5.1 projectmanager-app/src/test/java/com/projectmanagement/app Directory Reference

Files

- file MainTest.java

5.2 projectmanager-app/src/test/java/com Directory Reference

Directories

- directory projectmanagement

5.3 projectmanager-app/src/test/java Directory Reference

Directories

- directory com

5.4 projectmanager-app/src/test/java/com/projectmanagement/lib Directory Reference

Directories

- directory models
- directory services
- directory storage

5.5 projectmanager-app/src/test/java/com/projectmanagement/lib/↩ models Directory Reference

Files

- file ProjectTest.java
- file TaskTest.java
- file UserTest.java

5.6 projectmanager-app/src/test/java/com/projectmanagement Directory Reference

Directories

- directory app
- directory lib

5.7 projectmanager-app Directory Reference

Directories

- directory src

5.8 projectmanager-app/src/test/java/com/projectmanagement/lib/↩ services Directory Reference

Files

- file ProjectServiceTest.java
- file ReportServiceTest.java
- file TaskServiceTest.java
- file UserServiceTest.java

5.9 projectmanager-app/src Directory Reference

Directories

- directory test

5.10 projectmanager-app/src/test/java/com/projectmanagement/lib/storage Directory Reference

Files

- file BinaryRepositoryTest.java
- file MySQLRepositoryTest.java
- file RepositoryFactoryTest.java
- file SQLiteRepositoryTest.java

5.11 projectmanager-app/src/test Directory Reference

Directories

- directory java

Chapter 6

Namespace Documentation

6.1 Package com.projectmanagement.app

Classes

- class MainTest

6.2 Package com.projectmanagement.lib.models

Classes

- class ProjectTest
- class TaskTest
- class UserTest

6.3 Package com.projectmanagement.lib.services

Classes

- class ProjectServiceTest
- class ReportServiceTest
- class TaskServiceTest
- class UserServiceTest

6.4 Package com.projectmanagement.lib.storage

Classes

- class **BinaryRepositoryTest**
- class **MySQLRepositoryTest**
- class RepositoryFactoryTest
- class **SQLiteRepositoryTest**

Chapter 7

Class Documentation

7.1 com.projectmanagement.app.MainTest Class Reference

Public Member Functions

- void setUpOutput ()
- void restoreSystemInputOutput ()
- void testMain_GuestModeAndExit () throws Exception
- void testMain_FullNavigation () throws Exception

7.1.1 Detailed Description

Definition at line 13 of file MainTest.java.

7.1.2 Member Function Documentation

7.1.2.1 restoreSystemInputOutput()

```
void com.projectmanagement.app.MainTest.restoreSystemInputOutput ()
```

Definition at line 36 of file MainTest.java.

```
00036                                         {
00037         System.setIn(systemIn);
00038         System.setOut(systemOut);
00039     }
```

7.1.2.2 setUpOutput()

```
void com.projectmanagement.app.MainTest.setUpOutput ()
```

Definition at line 20 of file MainTest.java.

```
00020                                 {
00021         testOut = new ByteArrayOutputStream();
00022         System.setOut(new PrintStream(testOut));
00023
00024         // Clear all storage files to ensure tests run in a clean, isolated environment
00025         java.io.File usersFile = new java.io.File("users.dat");
00026         if(usersFile.exists()) usersFile.delete();
00027         java.io.File projectsFile = new java.io.File("projects.dat");
00028         if(projectsFile.exists()) projectsFile.delete();
00029         java.io.File tasksFile = new java.io.File("tasks.dat");
00030         if(tasksFile.exists()) tasksFile.delete();
00031         java.io.File dbFile = new java.io.File("projectmanager.db");
00032         if(dbFile.exists()) dbFile.delete();
00033     }
```

7.1.2.3 testMain_FullNavigation()

```
void com.projectmanagement.app.MainTest.testMain_FullNavigation () throws Exception
```

Definition at line 60 of file MainTest.java.

```
00060                                                    {
00061         // Reset storage files by swapping to BINARY and back so we can cleanly test empty states
00062         // In this test environment, we'll try to just cover the cases.
00063         // But since we want empty lists, we might already have data from previous tests.
00064         // We will do both valid and invalid paths.
00065         StringBuilder sb = new StringBuilder();
00066
00067         // --- AUTH MENU ---
00068         // 1. Try invalid input (letters) -> "Invalid character! Please enter numbers only."
00069         sb.append("a\n");
00070         // 2. Try invalid range
00071         sb.append("9\n");
00072
00073         // Try Login before register
00074         sb.append("1\n");
00075         sb.append("nonexistent\n");
00076         sb.append("pass\n");
00077         sb.append("\n"); // pause
00078
00079         // Register (User1)
00080         sb.append("2\n");
00081         sb.append("usr\n");
00082         sb.append("email@email.com\n");
00083         sb.append("pass\n");
00084         sb.append("\n"); // pause
00085
00086         // Register again (User1) to hit the catch block
00087         sb.append("2\n");
00088         sb.append("usr\n");
00089         sb.append("email2@email.com\n");
00090         sb.append("pass2\n");
00091         sb.append("\n"); // pause
00092
00093         // Login correct
00094         sb.append("1\n");
00095         sb.append("usr\n");
00096         sb.append("pass\n");
00097         sb.append("\n"); // pause
00098
00099         // --- MAIN MENU (We are logged in) ---
00100
00101         // Test empty views
00102         // Project Setup -> View all (Empty)
00103         sb.append("1\n");
00104         sb.append("2\n");
00105         sb.append("\n"); // pause
00106         sb.append("3\n"); // Back
00107
00108         // Task Assignment -> Empty checks
00109         sb.append("2\n");
00110         sb.append("1\n"); // Create Task (No projects error)
00111         sb.append("\n"); // pause
00112         sb.append("2\n"); // Assign User (No tasks error)
00113         sb.append("\n"); // pause
00114         sb.append("3\n"); // View all Tasks (Empty)
00115         sb.append("\n"); // pause
00116         sb.append("4\n"); // Back
00117
00118         // Progress Tracking -> Empty
00119         sb.append("3\n");
00120         sb.append("1\n"); // Update Task
00121         sb.append("\n"); // pause
00122         sb.append("2\n"); // Back
00123
00124         // Now populate data
00125         // Project Setup
00126         sb.append("1\n");
00127         // Create project Invalid Name
00128         sb.append("1\n");
00129         sb.append("\n"); // Blank name -> Exception
00130         sb.append("\n"); // desc
00131         sb.append("\n"); // pause
00132
00133         sb.append("1\n"); // Create project Valid
00134         sb.append("ProjA\n"); // name
00135         sb.append("DescA\n"); // desc
00136         sb.append("\n"); // pause
00137
00138         // Project Setup -> View all (Has Project)
```

```
        sb.append("2\n");
        sb.append("\n"); // pause
        sb.append("3\n"); // Back

        // Task Assignment
        sb.append("2\n");
        // Add Task - Invalid project selection
        sb.append("1\n");
        sb.append("99\n"); // Invalid project index
        sb.append("\n"); // pause
        // Add Task - Invalid input format
        sb.append("1\n");
        sb.append("abc\n"); // Invalid index format
        sb.append("\n"); // pause
        // Add Task - Valid
        sb.append("1\n");
        sb.append("1\n"); // Select Project 1
        sb.append("TaskA\n"); // title
        sb.append("DescA\n"); // desc
        sb.append("\n"); // pause

        // Assign User - Invalid Number Format
        sb.append("2\n");
        sb.append("abc\n"); // Invalid task index format
        sb.append("1\n");
        sb.append("\n"); // pause
        // Assign User - Valid
        sb.append("2\n");
        sb.append("1\n"); // Task 1
        sb.append("1\n"); // User 1
        sb.append("\n"); // pause

        // View all Tasks
        sb.append("3\n");
        sb.append("\n"); // pause

        sb.append("4\n"); // Back

        // Progress Tracking
        sb.append("3\n");
        // Update Task - Invalid
        sb.append("1\n");
        sb.append("abc\n"); // Invalid
        sb.append("1\n");
        sb.append("\n"); // pause
        // Update Task - Valid
        sb.append("1\n");
        sb.append("1\n"); // Task 1
        sb.append("3\n"); // status 3 (DONE)
        sb.append("\n"); // pause

        sb.append("2\n"); // Back

        // Reporting
        sb.append("4\n");
        sb.append("1\n"); // General
        sb.append("\n"); // pause
        sb.append("2\n"); // Task
        sb.append("\n"); // pause
        sb.append("3\n"); // Back

        // Logout
        sb.append("5\n");
        sb.append("\n"); // pause

        // Auth Menu -> Swap Storage Cancel
        sb.append("4\n"); // Change storage
        sb.append("4\n"); // Cancel

        // Auth Menu -> Swap Storage
        sb.append("4\n"); // Change storage
        sb.append("1\n"); // Select Binary
        sb.append("\n"); // pause

        // Exit
        sb.append("5\n");

        provideInput(sb.toString());
        try { Main.main(new String[0]); } catch(Throwable t) {}
    }
```

7.1.2.4 testMain_GuestModeAndExit()

```
void com.projectmanagement.app.MainTest.testMain_GuestModeAndExit () throws Exception
```

Definition at line 48 of file MainTest.java.

```
00048                                                                {
00049         // App starts in Auth Menu
00050         // 3 -> Guest Mode
00051         // \n -> Pause
00052         // 6 -> Exit Application (From Main Menu)
00053         String simulatedUserInput = "3\n\n6\n";
00054         provideInput(simulatedUserInput);
00055
00056         try { Main.main(new String[0]); } catch(Throwable t) {}
00057     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/test/java/com/projectmanagement/app/MainTest.java

7.2 com.projectmanagement.lib.services.ProjectServiceTest Class Reference

Public Member Functions

- void setup ()
- void testCreateProject_Success ()
- void testCreateProject_NullInputs_ThrowsException ()
- void testAddTaskToProject_Success ()
- void testAddTaskToProject_InvalidInputs_ThrowsException ()
- void testGetAllProjects ()

7.2.1 Detailed Description

Definition at line 16 of file ProjectServiceTest.java.

7.2.2 Member Function Documentation

7.2.2.1 setup()

```
void com.projectmanagement.lib.services.ProjectServiceTest.setup ()
```

Definition at line 24 of file ProjectServiceTest.java.

```
00024                         {
00025         projectService = new ProjectService(projectRepositoryMock);
00026     }
```

7.2.2.2 testAddTaskToProject_InvalidInputs_ThrowsException()

```
void com.projectmanagement.lib.services.ProjectServiceTest.testAddTaskToProject_Invalid↩
Inputs_ThrowsException ()
```

Definition at line 75 of file ProjectServiceTest.java.

```
00075                                                                {
00076         assertThrows(IllegalArgumentException.class, () -> {
00077             projectService.addTaskToProject(null, new Task());
00078         });
00079
00080         assertThrows(IllegalArgumentException.class, () -> {
00081             projectService.addTaskToProject("proj-id", null);
00082         });
00083
00084         // Test non-existent project
00085         when(projectRepositoryMock.findById("missing-proj")).thenReturn(null);
00086         assertThrows(IllegalArgumentException.class, () -> {
00087             projectService.addTaskToProject("missing-proj", new Task());
00088         });
00089
00090         verify(projectRepositoryMock, never()).update(any(Project.class));
00091     }
```

7.2.2.3 testAddTaskToProject_Success()

```
void com.projectmanagement.lib.services.ProjectServiceTest.testAddTaskToProject_Success ()
```

Definition at line 60 of file ProjectServiceTest.java.

```
00060                                                {
00061         String projectId = "proj-1";
00062         Project mockedProject = new Project(projectId, "Test Project", "Desc");
00063         Task newTask = new Task("task-1", "A Task", "Doing it");
00064
00065         when(projectRepositoryMock.findById(projectId)).thenReturn(mockedProject);
00066
00067         projectService.addTaskToProject(projectId, newTask);
00068
00069         // Project should now have 1 task
00070         assertEquals(1, mockedProject.getTasks().size());
00071         verify(projectRepositoryMock, times(1)).update(mockedProject);
00072     }
```

7.2.2.4 testCreateProject_NullInputs_ThrowsException()

```
void com.projectmanagement.lib.services.ProjectServiceTest.testCreateProject_NullInputs_↩
ThrowsException ()
```

Definition at line 39 of file ProjectServiceTest.java.

```
00039                                                                {
00040         assertThrows(IllegalArgumentException.class, () -> {
00041             projectService.createProject(null, "Name", "Desc");
00042         });
00043
00044         assertThrows(IllegalArgumentException.class, () -> {
00045             projectService.createProject("ID", null, "Desc");
00046         });
00047
00048         assertThrows(IllegalArgumentException.class, () -> {
00049             projectService.createProject("ID", "", "Desc"); // Empty string
00050         });
00051
00052         assertThrows(IllegalArgumentException.class, () -> {
00053             projectService.createProject("ID", "Name", null); // null desc
00054         });
00055
00056         verify(projectRepositoryMock, never()).save(any(Project.class));
00057     }
```

7.2.2.5 testCreateProject_Success()

```
void com.projectmanagement.lib.services.ProjectServiceTest.testCreateProject_Success ()
```

Definition at line 29 of file ProjectServiceTest.java.

```
00029                                                {
00030         String id = "proj-1";
00031         String name = "Test Project";
00032         String desc = "Testing Project Logic";
00033
00034         projectService.createProject(id, name, desc);
00035         verify(projectRepositoryMock, times(1)).save(any(Project.class));
00036     }
```

7.2.2.6 testGetAllProjects()

```
void com.projectmanagement.lib.services.ProjectServiceTest.testGetAllProjects ()
```

Definition at line 94 of file ProjectServiceTest.java.

```
00094                                                {
00095         java.util.List<Project> mockList = java.util.Arrays.asList(new Project("proj-1", "T", "D"));
00096         when(projectRepositoryMock.findAll()).thenReturn(mockList);
00097         java.util.List<Project> result = projectService.getAllProjects();
00098         assertEquals(1, result.size());
00099         verify(projectRepositoryMock, times(1)).findAll();
00100     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/test/java/com/projectmanagement/lib/services/ProjectServiceTest.java

7.3 com.projectmanagement.lib.models.ProjectTest Class Reference

Public Member Functions

- void setUp ()
- void tearDown ()
- void testDefaultConstructor ()
- void testParameterizedConstructor ()
- void testSettersAndGetters ()
- void testAddRemoveTask ()
- void testDisplayDetails ()

7.3.1 Detailed Description

Definition at line 14 of file ProjectTest.java.

7.3.2 Member Function Documentation

7.3.2.1 setUp()

```
void com.projectmanagement.lib.models.ProjectTest.setUp ()
```

Definition at line 20 of file ProjectTest.java.

```
00020                         {
00021         System.setOut(new PrintStream(outContent));
00022     }
```

7.3.2.2 tearDown()

```
void com.projectmanagement.lib.models.ProjectTest.tearDown ()
```

Definition at line 25 of file ProjectTest.java.

```
00025                           {
00026         System.setOut(originalOut);
00027     }
```

7.3.2.3 testAddRemoveTask()

```
void com.projectmanagement.lib.models.ProjectTest.testAddRemoveTask ()
```

Definition at line 68 of file ProjectTest.java.

```
00068                                   {
00069         Project project = new Project();
00070         Task task1 = new Task("t1", "T1", "Desc1");
00071         Task task2 = new Task("t2", "T2", "Desc2");
00072
00073         project.addTask(task1);
00074         project.addTask(task2);
00075         assertEquals(2, project.getTasks().size());
00076         assertTrue(project.getTasks().contains(task1));
00077
00078         project.removeTask(task1);
00079         assertEquals(1, project.getTasks().size());
00080         assertFalse(project.getTasks().contains(task1));
00081     }
```

7.3.2.4 testDefaultConstructor()

```
void com.projectmanagement.lib.models.ProjectTest.testDefaultConstructor ()
```

Definition at line 30 of file ProjectTest.java.

```
00030                                    {
00031         Project project = new Project();
00032         assertNull(project.getId());
00033         assertNotNull(project.getCreatedDate());
00034         assertNotNull(project.getTasks());
00035         assertTrue(project.getTasks().isEmpty());
00036     }
```

7.3.2.5 testDisplayDetails()

```
void com.projectmanagement.lib.models.ProjectTest.testDisplayDetails ()
```

Definition at line 84 of file ProjectTest.java.

```
00084                                    {
00085         Project project = new Project("p3", "Gamma", "Proj Gamma");
00086         project.addTask(new Task("t1", "T1", "D1"));
00087         project.displayDetails();
00088         String output = outContent.toString();
00089         assertTrue(output.contains("Project ID: p3"));
00090         assertTrue(output.contains("Name: Gamma"));
00091         assertTrue(output.contains("Description: Proj Gamma"));
00092         assertTrue(output.contains("Total Tasks: 1"));
00093     }
```

7.3.2.6 testParameterizedConstructor()

```
void com.projectmanagement.lib.models.ProjectTest.testParameterizedConstructor ()
```

Definition at line 39 of file ProjectTest.java.

```
00039                                    {
00040         Project project = new Project("p1", "Alpha", "Project Alpha description");
00041         assertEquals("p1", project.getId());
00042         assertEquals("Alpha", project.getName());
00043         assertEquals("Project Alpha description", project.getDescription());
00044         assertNotNull(project.getTasks());
00045         assertTrue(project.getTasks().isEmpty());
00046     }
```

7.3.2.7 testSettersAndGetters()

```
void com.projectmanagement.lib.models.ProjectTest.testSettersAndGetters ()
```

Definition at line 49 of file ProjectTest.java.

```
00049                                    {
00050         Project project = new Project();
00051         project.setId("p2");
00052         project.setName("Beta");
00053         project.setDescription("Project Beta description");
00054
00055         List<Task> tasks = new ArrayList<>();
00056         Task task = new Task("t1", "T1", "Desc");
00057         tasks.add(task);
00058         project.setTasks(tasks);
00059
00060         assertEquals("p2", project.getId());
00061         assertEquals("Beta", project.getName());
00062         assertEquals("Project Beta description", project.getDescription());
00063         assertEquals(1, project.getTasks().size());
00064         assertEquals(task, project.getTasks().get(0));
00065     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/test/java/com/projectmanagement/lib/models/ProjectTest.java

7.4 com.projectmanagement.lib.services.ReportServiceTest Class Reference

Public Member Functions

- void setUp ()
- void testGenerateGeneralSystemReport ()
- void testGenerateGeneralSystemReport_Empty ()
- void testGenerateGeneralSystemReport_EmptyLists ()
- void testGenerateTaskStatusReport ()
- void testGenerateTaskStatusReport_Empty ()

7.4.1 Detailed Description

Definition at line 14 of file ReportServiceTest.java.

7.4.2 Member Function Documentation

7.4.2.1 setUp()

```
void com.projectmanagement.lib.services.ReportServiceTest.setUp ()
```

Definition at line 22 of file ReportServiceTest.java.

```
00022                        {
00023         mockProjectService = mock(ProjectService.class);
00024         mockTaskService = mock(TaskService.class);
00025         mockUserService = mock(UserService.class);
00026         reportService = new ReportService(mockProjectService, mockTaskService, mockUserService);
00027     }
```

7.4.2.2 testGenerateGeneralSystemReport()

```
void com.projectmanagement.lib.services.ReportServiceTest.testGenerateGeneralSystemReport ()
```

Definition at line 30 of file ReportServiceTest.java.

```
00030                                                {
00031         // Arrange
00032         User u1 = new User("u1", "U1", "u1@test.com", "pass");
00033         User u2 = new User("u2", "U2", "u2@test.com", "pass");
00034
00035         Project p1 = new Project("p1", "Project 1", "Desc");
00036         Task t1 = new Task("t1", "Task 1", "Desc");
00037         p1.addTask(t1);
00038
00039         when(mockUserService.getAllUsers()).thenReturn(Arrays.asList(u1, u2));
00040         when(mockProjectService.getAllProjects()).thenReturn(Arrays.asList(p1));
00041         when(mockTaskService.getAllTasks()).thenReturn(Arrays.asList(t1));
00042
00043         // Act & Assert (mostly to ensure no exceptions are thrown and it covers the lines)
00044         reportService.generateGeneralSystemReport();
00045     }
```

7.4.2.3 testGenerateGeneralSystemReport_Empty()

```
void com.projectmanagement.lib.services.ReportServiceTest.testGenerateGeneralSystemReport_↩
Empty ()
```

Definition at line 48 of file ReportServiceTest.java.

```
00048                                                    {
00049         when(mockUserService.getAllUsers()).thenReturn(null);
00050         when(mockProjectService.getAllProjects()).thenReturn(null);
00051         when(mockTaskService.getAllTasks()).thenReturn(null);
00052
00053         reportService.generateGeneralSystemReport();
00054     }
```

7.4.2.4 testGenerateGeneralSystemReport_EmptyLists()

```
void com.projectmanagement.lib.services.ReportServiceTest.testGenerateGeneralSystemReport_↩
EmptyLists ()
```

Definition at line 57 of file ReportServiceTest.java.

```
00057                                                          {
00058         when(mockUserService.getAllUsers()).thenReturn(new ArrayList<>());
00059         when(mockProjectService.getAllProjects()).thenReturn(new ArrayList<>());
00060         when(mockTaskService.getAllTasks()).thenReturn(new ArrayList<>());
00061         reportService.generateGeneralSystemReport();
00062     }
```

7.4.2.5 testGenerateTaskStatusReport()

```
void com.projectmanagement.lib.services.ReportServiceTest.testGenerateTaskStatusReport ()
```

Definition at line 65 of file ReportServiceTest.java.

```
00065                                             {
00066         Task t1 = new Task("t1", "T1", "Desc");
00067         t1.setStatus(TaskStatus.TODO);
00068
00069         Task t2 = new Task("t2", "T2", "Desc");
00070         t2.setStatus(TaskStatus.IN_PROGRESS);
00071
00072         Task t3 = new Task("t3", "T3", "Desc");
00073         t3.setStatus(TaskStatus.DONE);
00074
00075         when(mockTaskService.getAllTasks()).thenReturn(Arrays.asList(t1, t2, t3));
00076
00077         reportService.generateTaskStatusReport();
00078     }
```

7.4.2.6 testGenerateTaskStatusReport_Empty()

```
void com.projectmanagement.lib.services.ReportServiceTest.testGenerateTaskStatusReport_Empty
()
```

Definition at line 81 of file ReportServiceTest.java.

```
00081                                                    {
00082         when(mockTaskService.getAllTasks()).thenReturn(new ArrayList<>());
00083         reportService.generateTaskStatusReport();
00084
00085         when(mockTaskService.getAllTasks()).thenReturn(null);
00086         reportService.generateTaskStatusReport();
00087     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/test/java/com/projectmanagement/lib/services/ReportServiceTest.java

7.5 com.projectmanagement.lib.storage.RepositoryFactoryTest Class Reference

Public Member Functions

- void testGetRepository_BinaryStorage ()
- void testGetRepository_SQLiteStorage ()
- void testGetRepository_MySQLStorage ()
- void testGetRepository_UnsupportedStorageType ()

7.5.1 Detailed Description

Definition at line 10 of file RepositoryFactoryTest.java.

7.5.2 Member Function Documentation

7.5.2.1 testGetRepository_BinaryStorage()

```
void com.projectmanagement.lib.storage.RepositoryFactoryTest.testGetRepository_BinaryStorage
()
```

Definition at line 13 of file RepositoryFactoryTest.java.

```
00013                                                   {
00014         StorageConfig config = new StorageConfig(StorageType.BINARY_FILE);
00015         RepositoryFactory factory = new RepositoryFactory(config);
00016
00017         IRepository<User> useRepo = factory.getUserRepository();
00018         assertTrue(useRepo instanceof BinaryUserRepository);
00019
00020         IRepository<Project> projRepo = factory.getProjectRepository();
00021         assertTrue(projRepo instanceof BinaryProjectRepository);
00022
00023         IRepository<Task> taskRepo = factory.getTaskRepository();
00024         assertTrue(taskRepo instanceof BinaryTaskRepository);
00025     }
```

7.5.2.2 testGetRepository_MySQLStorage()

```
void com.projectmanagement.lib.storage.RepositoryFactoryTest.testGetRepository_MySQLStorage ()
```

Definition at line 43 of file RepositoryFactoryTest.java.

```
00043                                                   {
00044         StorageConfig config = new StorageConfig(StorageType.MYSQL);
00045         RepositoryFactory factory = new RepositoryFactory(config);
00046
00047         IRepository<User> useRepo = factory.getUserRepository();
00048         assertTrue(useRepo instanceof MySQLUserRepository);
00049
00050         IRepository<Project> projRepo = factory.getProjectRepository();
00051         assertTrue(projRepo instanceof MySQLProjectRepository);
00052
00053         IRepository<Task> taskRepo = factory.getTaskRepository();
00054         assertTrue(taskRepo instanceof MySQLTaskRepository);
00055     }
```

7.5.2.3 testGetRepository_SQLiteStorage()

```
void com.projectmanagement.lib.storage.RepositoryFactoryTest.testGetRepository_SQLiteStorage
()
```

Definition at line 28 of file RepositoryFactoryTest.java.

```
00028                                                   {
00029         StorageConfig config = new StorageConfig(StorageType.SQLITE);
00030         RepositoryFactory factory = new RepositoryFactory(config);
00031
00032         IRepository<User> useRepo = factory.getUserRepository();
00033         assertTrue(useRepo instanceof SQLiteUserRepository);
00034
00035         IRepository<Project> projRepo = factory.getProjectRepository();
00036         assertTrue(projRepo instanceof SQLiteProjectRepository);
00037
00038         IRepository<Task> taskRepo = factory.getTaskRepository();
00039         assertTrue(taskRepo instanceof SQLiteTaskRepository);
00040     }
```

7.5.2.4 testGetRepository_UnsupportedStorageType()

```
void com.projectmanagement.lib.storage.RepositoryFactoryTest.testGetRepository_Unsupported↩
StorageType ()
```

Definition at line 58 of file RepositoryFactoryTest.java.

```
00058                                                           {
00059         // Technically Java enums enforce type safety so this won't happen normally,
00060         // but it's good practice to ensure the switch loop handles default fallthrough errors.
00061
00062         // Simulating the edge case where config gets set to null (or someone adds a new enum type but
      forgets to add it to the factory switch cases)
00063         StorageConfig config = new StorageConfig(null);
00064         RepositoryFactory factory = new RepositoryFactory(config);
00065
00066         assertThrows(IllegalStateException.class, factory::getUserRepository);
00067         assertThrows(IllegalStateException.class, factory::getProjectRepository);
00068         assertThrows(IllegalStateException.class, factory::getTaskRepository);
00069     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/test/java/com/projectmanagement/lib/storage/RepositoryFactoryTest.java

7.6 com.projectmanagement.lib.services.TaskServiceTest Class Reference

Public Member Functions

- void setup ()
- void testRegisterTask_Success ()
- void testRegisterTask_NullFields_ThrowsException ()
- void testAssignTaskToUser_Success ()
- void testAssignTaskToUser_InvalidInputs_ThrowsException ()
- void testUpdateTaskStatus_Success ()
- void testUpdateTaskStatus_InvalidInputs_ThrowsException ()
- void testGetAllTasks ()

7.6.1 Detailed Description

Definition at line 17 of file TaskServiceTest.java.

7.6.2 Member Function Documentation

7.6.2.1 setup()

```
void com.projectmanagement.lib.services.TaskServiceTest.setup ()
```

Definition at line 25 of file TaskServiceTest.java.

```
00025                         {
00026         taskService = new TaskService(taskRepositoryMock);
00027     }
```

7.6.2.2 testAssignTaskToUser_InvalidInputs_ThrowsException()

```
void com.projectmanagement.lib.services.TaskServiceTest.testAssignTaskToUser_InvalidInputs_↩
ThrowsException ()
```

Definition at line 75 of file TaskServiceTest.java.

```
00075                                                                   {
00076         assertThrows(IllegalArgumentException.class, () -> {
00077             taskService.assignTaskToUser(null, new User());
00078         });
00079
00080         assertThrows(IllegalArgumentException.class, () -> {
00081             taskService.assignTaskToUser("task-1", null);
00082         });
00083
00084         when(taskRepositoryMock.findById("missing-task")).thenReturn(null);
00085         assertThrows(IllegalArgumentException.class, () -> {
00086             taskService.assignTaskToUser("missing-task", new User());
00087         });
00088
00089         verify(taskRepositoryMock, never()).update(any(Task.class));
00090     }
```

7.6.2.3 testAssignTaskToUser_Success()

```
void com.projectmanagement.lib.services.TaskServiceTest.testAssignTaskToUser_Success ()
```

Definition at line 61 of file TaskServiceTest.java.

```
00061                                           {
00062         String taskId = "task-1";
00063         Task trackingTask = new Task(taskId, "Title", "Desc");
00064         User assignedUser = new User("user-1", "Bob", "bob@user.com", "hash");
00065
00066         when(taskRepositoryMock.findById(taskId)).thenReturn(trackingTask);
00067
00068         taskService.assignTaskToUser(taskId, assignedUser);
00069
00070         assertEquals(assignedUser, trackingTask.getAssignedUser());
00071         verify(taskRepositoryMock, times(1)).update(trackingTask);
00072     }
```

7.6.2.4 testGetAllTasks()

```
void com.projectmanagement.lib.services.TaskServiceTest.testGetAllTasks ()
```

Definition at line 124 of file TaskServiceTest.java.

```
00124                                   {
00125         java.util.List<Task> mockList = java.util.Arrays.asList(new Task("task-1", "T", "D"));
00126         when(taskRepositoryMock.findAll()).thenReturn(mockList);
00127         java.util.List<Task> result = taskService.getAllTasks();
00128         assertEquals(1, result.size());
00129         verify(taskRepositoryMock, times(1)).findAll();
00130     }
```

7.6.2.5 testRegisterTask_NullFields_ThrowsException()

```
void com.projectmanagement.lib.services.TaskServiceTest.testRegisterTask_NullFields_Throws↩
Exception ()
```

Definition at line 40 of file TaskServiceTest.java.

```
00040                                                           {
00041         assertThrows(IllegalArgumentException.class, () -> {
00042             taskService.registerTask(null, "Title", "Desc");
00043         });
00044
00045         assertThrows(IllegalArgumentException.class, () -> {
00046             taskService.registerTask("ID", null, "Desc");
00047         });
00048
00049         assertThrows(IllegalArgumentException.class, () -> {
00050             taskService.registerTask("ID", "", "Desc"); // Empty title
00051         });
00052
00053         assertThrows(IllegalArgumentException.class, () -> {
00054             taskService.registerTask("ID", "Title", null);
00055         });
00056
00057         verify(taskRepositoryMock, never()).save(any(Task.class));
00058     }
```

7.6.2.6 testRegisterTask_Success()

```
void com.projectmanagement.lib.services.TaskServiceTest.testRegisterTask_Success ()
```

Definition at line 30 of file TaskServiceTest.java.

```
00030                                           {
00031         String id = "task-1";
00032         String title = "Complete Documentation";
00033         String desc = "Write all the missing Doxygen comments";
00034
00035         taskService.registerTask(id, title, desc);
00036         verify(taskRepositoryMock, times(1)).save(any(Task.class));
00037     }
```

7.6.2.7 testUpdateTaskStatus_InvalidInputs_ThrowsException()

```
void com.projectmanagement.lib.services.TaskServiceTest.testUpdateTaskStatus_InvalidInputs_↩
ThrowsException ()
```

Definition at line 106 of file TaskServiceTest.java.

```
00106                                                                   {
00107         assertThrows(IllegalArgumentException.class, () -> {
00108             taskService.updateTaskStatus(null, TaskStatus.DONE);
00109         });
00110
00111         assertThrows(IllegalArgumentException.class, () -> {
00112             taskService.updateTaskStatus("task-1", null);
00113         });
00114
00115         when(taskRepositoryMock.findById("missing-task")).thenReturn(null);
00116         assertThrows(IllegalArgumentException.class, () -> {
00117              taskService.updateTaskStatus("missing-task", TaskStatus.IN_PROGRESS);
00118         });
00119
00120         verify(taskRepositoryMock, never()).update(any(Task.class));
00121     }
```

7.6.2.8 testUpdateTaskStatus_Success()

```
void com.projectmanagement.lib.services.TaskServiceTest.testUpdateTaskStatus_Success ()
```

Definition at line 93 of file TaskServiceTest.java.

```
00093                                                {
00094         String taskId = "task-1";
00095         Task operatingTask = new Task(taskId, "Title", "Desc");
00096
00097         when(taskRepositoryMock.findById(taskId)).thenReturn(operatingTask);
00098
00099         taskService.updateTaskStatus(taskId, TaskStatus.IN_PROGRESS);
00100
00101         assertEquals(TaskStatus.IN_PROGRESS, operatingTask.getStatus());
00102         verify(taskRepositoryMock, times(1)).update(operatingTask);
00103     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/test/java/com/projectmanagement/lib/services/TaskServiceTest.java

7.7 com.projectmanagement.lib.models.TaskTest Class Reference

Public Member Functions

- void setUp ()
- void tearDown ()
- void testDefaultConstructor ()
- void testParameterizedConstructor ()
- void testSettersAndGetters ()
- void testDisplayDetailsUnassigned ()
- void testDisplayDetailsAssigned ()

7.7.1 Detailed Description

Definition at line 12 of file TaskTest.java.

7.7.2 Member Function Documentation

7.7.2.1 setUp()

```
void com.projectmanagement.lib.models.TaskTest.setUp ()
```

Definition at line 18 of file TaskTest.java.

```
00018                         {
00019         System.setOut(new PrintStream(outContent));
00020     }
```

7.7.2.2 tearDown()

```
void com.projectmanagement.lib.models.TaskTest.tearDown ()
```

Definition at line 23 of file TaskTest.java.

```
00023                             {
00024         System.setOut(originalOut);
00025     }
```

7.7.2.3 testDefaultConstructor()

```
void com.projectmanagement.lib.models.TaskTest.testDefaultConstructor ()
```

Definition at line 28 of file TaskTest.java.

```
00028                                        {
00029         Task task = new Task();
00030         assertNull(task.getId());
00031         assertNotNull(task.getCreatedDate());
00032         assertEquals(TaskStatus.TODO, task.getStatus());
00033         assertNull(task.getTitle());
00034         assertNull(task.getDescription());
00035         assertNull(task.getAssignedUser());
00036     }
```

7.7.2.4 testDisplayDetailsAssigned()

```
void com.projectmanagement.lib.models.TaskTest.testDisplayDetailsAssigned ()
```

Definition at line 77 of file TaskTest.java.

```
00077                                                {
00078         Task task = new Task("t4", "Write tests", "Increase coverage");
00079         task.setStatus(TaskStatus.DONE);
00080         User user = new User("u1", "charlie", "charlie@m.com", "pass");
00081         task.setAssignedUser(user);
00082         task.displayDetails();
00083         String output = outContent.toString();
00084         assertTrue(output.contains("Task ID: t4"));
00085         assertTrue(output.contains("Title: Write tests"));
00086         assertTrue(output.contains("Status: DONE"));
00087         assertTrue(output.contains("Assigned To: charlie"));
00088     }
```

7.7.2.5 testDisplayDetailsUnassigned()

```
void com.projectmanagement.lib.models.TaskTest.testDisplayDetailsUnassigned ()
```

Definition at line 66 of file TaskTest.java.

```
00066                                                {
00067         Task task = new Task("t3", "Review code", "Review PR #42");
00068         task.displayDetails();
00069         String output = outContent.toString();
00070         assertTrue(output.contains("Task ID: t3"));
00071         assertTrue(output.contains("Title: Review code"));
00072         assertTrue(output.contains("Status: TODO"));
00073         assertTrue(output.contains("Assigned To: Unassigned"));
00074     }
```

7.7.2.6 testParameterizedConstructor()

```
void com.projectmanagement.lib.models.TaskTest.testParameterizedConstructor ()
```

Definition at line 39 of file TaskTest.java.

```
00039                                                {
00040         Task task = new Task("t1", "Fix bug", "Fix the critical issue");
00041         assertEquals("t1", task.getId());
00042         assertEquals("Fix bug", task.getTitle());
00043         assertEquals("Fix the critical issue", task.getDescription());
00044         assertEquals(TaskStatus.TODO, task.getStatus());
00045         assertNull(task.getAssignedUser());
00046     }
```

7.7.2.7 testSettersAndGetters()

```
void com.projectmanagement.lib.models.TaskTest.testSettersAndGetters ()
```

Definition at line 49 of file TaskTest.java.

```
00049                                        {
00050         Task task = new Task();
00051         User user = new User("u1", "bob", "bob@example.com", "pass");
00052         task.setId("t2");
00053         task.setTitle("Develop feature");
00054         task.setDescription("Code the feature");
00055         task.setStatus(TaskStatus.IN_PROGRESS);
00056         task.setAssignedUser(user);
00057
00058         assertEquals("t2", task.getId());
00059         assertEquals("Develop feature", task.getTitle());
00060         assertEquals("Code the feature", task.getDescription());
00061         assertEquals(TaskStatus.IN_PROGRESS, task.getStatus());
00062         assertEquals(user, task.getAssignedUser());
00063     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/test/java/com/projectmanagement/lib/models/TaskTest.java

7.8 com.projectmanagement.lib.services.UserServiceTest Class Reference

Public Member Functions

- void setup ()
- void testRegisterUser_Success ()
- void testRegisterUser_NullFields_ThrowsException ()
- void testLogin_Success () throws Exception
- void testLogin_InvalidCredentials () throws Exception
- void testLogin_NullOrEmptyInputs ()
- void testLogin_NoUsersInDb ()
- void testGetAllUsers ()

7.8.1 Detailed Description

Definition at line 18 of file UserServiceTest.java.

7.8.2 Member Function Documentation

7.8.2.1 setup()

```
void com.projectmanagement.lib.services.UserServiceTest.setup ()
```

Definition at line 26 of file UserServiceTest.java.

```
00026                         {
00027         userService = new UserService(userRepositoryMock);
00028     }
```

7.8.2.2 testGetAllUsers()

```
void com.projectmanagement.lib.services.UserServiceTest.testGetAllUsers ()
```

Definition at line 125 of file UserServiceTest.java.

```
00125                                   {
00126         java.util.List<User> mockList = Arrays.asList(new User("u-1", "T", "D", "P"));
00127         when(userRepositoryMock.findAll()).thenReturn(mockList);
00128         java.util.List<User> result = userService.getAllUsers();
00129         assertEquals(1, result.size());
00130         verify(userRepositoryMock, times(1)).findAll();
00131     }
```

7.8.2.3 testLogin_InvalidCredentials()

```
void com.projectmanagement.lib.services.UserServiceTest.testLogin_InvalidCredentials () throws
Exception
```

Definition at line 95 of file UserServiceTest.java.

```
00095                                                                   {
00096         String username = "testuser";
00097         String plainPassword = "SuperSecretPassword";
00098         String wrongPassword = "WrongPassword!";
00099
00100         userService.registerUser("uuid-1", username, "test@user.com", plainPassword);
00101         ArgumentCaptor<User> userCaptor = ArgumentCaptor.forClass(User.class);
00102         verify(userRepositoryMock, times(1)).save(userCaptor.capture());
00103         User registeredUser = userCaptor.getValue();
00104
00105         when(userRepositoryMock.findAll()).thenReturn(Arrays.asList(registeredUser));
00106
00107         User loggedInUser = userService.login(username, wrongPassword);
00108         assertNull(loggedInUser, "Should return null for wrong password.");
00109     }
```

7.8.2.4 testLogin_NoUsersInDb()

```
void com.projectmanagement.lib.services.UserServiceTest.testLogin_NoUsersInDb ()
```

Definition at line 119 of file UserServiceTest.java.

```
00119                                   {
00120         when(userRepositoryMock.findAll()).thenReturn(null);
00121         assertNull(userService.login("user", "pass"));
00122     }
```

7.8.2.5 testLogin_NullOrEmptyInputs()

```
void com.projectmanagement.lib.services.UserServiceTest.testLogin_NullOrEmptyInputs ()
```

Definition at line 112 of file UserServiceTest.java.

```
00112                                           {
00113         assertNull(userService.login(null, "password"));
00114         assertNull(userService.login("username", null));
00115         assertNull(userService.login("username", ""));
00116     }
```

7.8.2.6 testLogin_Success()

```
void com.projectmanagement.lib.services.UserServiceTest.testLogin_Success () throws Exception
```

Definition at line 76 of file UserServiceTest.java.

```
00076                                                   {
00077         String username = "testuser";
00078         String plainPassword = "SuperSecretPassword";
00079
00080         // Register to capture correctly hashed user
00081         userService.registerUser("uuid-1", username, "test@user.com", plainPassword);
00082         ArgumentCaptor<User> userCaptor = ArgumentCaptor.forClass(User.class);
00083         verify(userRepositoryMock, times(1)).save(userCaptor.capture());
00084         User registeredUser = userCaptor.getValue();
00085
00086         when(userRepositoryMock.findAll()).thenReturn(Arrays.asList(registeredUser));
00087
00088         User loggedInUser = userService.login(username, plainPassword);
00089
00090         assertNotNull(loggedInUser);
00091         assertEquals(username, loggedInUser.getUsername());
00092     }
```

7.8.2.7 testRegisterUser_NullFields_ThrowsException()

```
void com.projectmanagement.lib.services.UserServiceTest.testRegisterUser_NullFields_Throws↩
Exception ()
```

Definition at line 50 of file UserServiceTest.java.

```
00050                                                       {
00051         assertThrows(IllegalArgumentException.class, () -> {
00052             userService.registerUser(null, "username", "email", "password");
00053         });
00054
00055         assertThrows(IllegalArgumentException.class, () -> {
00056             userService.registerUser("id", null, "email", "password");
00057         });
00058
00059         assertThrows(IllegalArgumentException.class, () -> {
00060             userService.registerUser("id", "username", null, "password");
00061         });
00062
00063         assertThrows(IllegalArgumentException.class, () -> {
00064             userService.registerUser("id", "username", "email", ""); // Empty string
00065         });
00066
00067         assertThrows(IllegalArgumentException.class, () -> {
00068             userService.registerUser("id", "username", "email", null); // null password
00069         });
00070
00071         // Verify create was NEVER called
00072         verify(userRepositoryMock, never()).save(any(User.class));
00073     }
```

7.8.2.8 testRegisterUser_Success()

```
void com.projectmanagement.lib.services.UserServiceTest.testRegisterUser_Success ()
```

Definition at line 31 of file UserServiceTest.java.

```
00031                                           {
00032         String id = "uuid-1";
00033         String username = "testuser";
00034         String email = "test@user.com";
00035         String plainPassword = "SuperSecretPassword";
00036
00037         userService.registerUser(id, username, email, plainPassword);
00038
00039         // Capture what was passed to create()
00040         ArgumentCaptor<User> userCaptor = ArgumentCaptor.forClass(User.class);
00041         verify(userRepositoryMock, times(1)).save(userCaptor.capture());
00042
```

```
        User capturedUser = userCaptor.getValue();
        assertEquals(username, capturedUser.getUsername());
        assertEquals(email, capturedUser.getEmail());
        assertNotEquals(plainPassword, capturedUser.getPassword(), "Password must be hashed, not plaintext!");
    }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/test/java/com/projectmanagement/lib/services/UserServiceTest.java

7.9 com.projectmanagement.lib.models.UserTest Class Reference

Public Member Functions

- void setUp ()
- void tearDown ()
- void testDefaultConstructor ()
- void testParameterizedConstructor ()
- void testSetters ()
- void testDisplayDetails ()

7.9.1 Detailed Description

Definition at line 14 of file UserTest.java.

7.9.2 Member Function Documentation

7.9.2.1 setUp()

```
void com.projectmanagement.lib.models.UserTest.setUp ()
```

Definition at line 20 of file UserTest.java.

```
                    {
        System.setOut(new PrintStream(outContent));
    }
```

7.9.2.2 tearDown()

```
void com.projectmanagement.lib.models.UserTest.tearDown ()
```

Definition at line 25 of file UserTest.java.

```
                        {
        System.setOut(originalOut);
    }
```

7.9.2.3 testDefaultConstructor()

```
void com.projectmanagement.lib.models.UserTest.testDefaultConstructor ()
```

Definition at line 30 of file UserTest.java.

```
00030                                        {
00031         User user = new User();
00032         assertNull(user.getId());
00033         assertNotNull(user.getCreatedDate());
00034         assertNull(user.getUsername());
00035         assertNull(user.getEmail());
00036         assertNull(user.getPassword());
00037     }
```

7.9.2.4 testDisplayDetails()

```
void com.projectmanagement.lib.models.UserTest.testDisplayDetails ()
```

Definition at line 67 of file UserTest.java.

```
00067                                       {
00068         User user = new User("u3", "alice", "alice@example.com", "pass");
00069         user.displayDetails();
00070         String output = outContent.toString();
00071         assertTrue(output.contains("User ID: u3"));
00072         assertTrue(output.contains("Username: alice"));
00073         assertTrue(output.contains("Email: alice@example.com"));
00074     }
```

7.9.2.5 testParameterizedConstructor()

```
void com.projectmanagement.lib.models.UserTest.testParameterizedConstructor ()
```

Definition at line 40 of file UserTest.java.

```
00040                                                {
00041         User user = new User("u1", "johndoe", "john@example.com", "pass123");
00042         assertEquals("u1", user.getId());
00043         assertEquals("johndoe", user.getUsername());
00044         assertEquals("john@example.com", user.getEmail());
00045         assertEquals("pass123", user.getPassword());
00046         assertNotNull(user.getCreatedDate());
00047     }
```

7.9.2.6 testSetters()

```
void com.projectmanagement.lib.models.UserTest.testSetters ()
```

Definition at line 50 of file UserTest.java.

```
00050                                 {
00051         User user = new User();
00052         user.setId("u2");
00053         user.setUsername("janedoe");
00054         user.setEmail("jane@example.com");
00055         user.setPassword("newpass");
00056         LocalDateTime now = LocalDateTime.now();
00057         user.setCreatedDate(now);
00058
00059         assertEquals("u2", user.getId());
00060         assertEquals("janedoe", user.getUsername());
00061         assertEquals("jane@example.com", user.getEmail());
00062         assertEquals("newpass", user.getPassword());
00063         assertEquals(now, user.getCreatedDate());
00064     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/test/java/com/projectmanagement/lib/models/UserTest.java

Chapter 8

File Documentation

8.1 projectmanager-app/src/test/java/com/projectmanagement/app/↩ MainTest.java File Reference

```
import java.util.Scanner;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.InputStream;
import java.io.PrintStream;
```

Classes

- class com.projectmanagement.app.MainTest

Packages

- package com.projectmanagement.app

8.2 MainTest.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.app;
00002
00003 import java.util.Scanner;
00004 import org.junit.jupiter.api.AfterEach;
00005 import org.junit.jupiter.api.BeforeEach;
00006 import org.junit.jupiter.api.Test;
00007
00008 import java.io.ByteArrayInputStream;
00009 import java.io.ByteArrayOutputStream;
00010 import java.io.InputStream;
00011 import java.io.PrintStream;
00012
00013 public class MainTest {
00014
```

```java
    private final InputStream systemIn = System.in;
    private final PrintStream systemOut = System.out;
    private ByteArrayOutputStream testOut;

    @BeforeEach
    public void setUpOutput() {
        testOut = new ByteArrayOutputStream();
        System.setOut(new PrintStream(testOut));

        // Clear all storage files to ensure tests run in a clean, isolated environment
        java.io.File usersFile = new java.io.File("users.dat");
        if(usersFile.exists()) usersFile.delete();
        java.io.File projectsFile = new java.io.File("projects.dat");
        if(projectsFile.exists()) projectsFile.delete();
        java.io.File tasksFile = new java.io.File("tasks.dat");
        if(tasksFile.exists()) tasksFile.delete();
        java.io.File dbFile = new java.io.File("projectmanager.db");
        if(dbFile.exists()) dbFile.delete();
    }

    @AfterEach
    public void restoreSystemInputOutput() {
        System.setIn(systemIn);
        System.setOut(systemOut);
    }

    private void provideInput(String data) {
        ByteArrayInputStream testIn = new ByteArrayInputStream(data.getBytes());
        System.setIn(testIn);
        // Reflection removed! Main now dynamically grabs scanner via Main.main()
    }

    @Test
    public void testMain_GuestModeAndExit() throws Exception {
        // App starts in Auth Menu
        // 3 -> Guest Mode
        // \n -> Pause
        // 6 -> Exit Application (From Main Menu)
        String simulatedUserInput = "3\n\n6\n";
        provideInput(simulatedUserInput);

        try { Main.main(new String[0]); } catch(Throwable t) {}
    }

    @Test
    public void testMain_FullNavigation() throws Exception {
        // Reset storage files by swapping to BINARY and back so we can cleanly test empty states
        // In this test environment, we'll try to just cover the cases.
        // But since we want empty lists, we might already have data from previous tests.
        // We will do both valid and invalid paths.
        StringBuilder sb = new StringBuilder();

        // --- AUTH MENU ---
        // 1. Try invalid input (letters) -> "Invalid character! Please enter numbers only."
        sb.append("a\n");
        // 2. Try invalid range
        sb.append("9\n");

        // Try Login before register
        sb.append("1\n");
        sb.append("nonexistent\n");
        sb.append("pass\n");
        sb.append("\n"); // pause

        // Register (User1)
        sb.append("2\n");
        sb.append("usr\n");
        sb.append("email@email.com\n");
        sb.append("pass\n");
        sb.append("\n"); // pause

        // Register again (User1) to hit the catch block
        sb.append("2\n");
        sb.append("usr\n");
        sb.append("email2@email.com\n");
        sb.append("pass2\n");
        sb.append("\n"); // pause

        // Login correct
        sb.append("1\n");
        sb.append("usr\n");
        sb.append("pass\n");
        sb.append("\n"); // pause

        // --- MAIN MENU (We are logged in) ---

        // Test empty views
```

```
        // Project Setup -> View all (Empty)
        sb.append("1\n");
        sb.append("2\n");
        sb.append("\n"); // pause
        sb.append("3\n"); // Back

        // Task Assignment -> Empty checks
        sb.append("2\n");
        sb.append("1\n"); // Create Task (No projects error)
        sb.append("\n"); // pause
        sb.append("2\n"); // Assign User (No tasks error)
        sb.append("\n"); // pause
        sb.append("3\n"); // View all Tasks (Empty)
        sb.append("\n"); // pause
        sb.append("4\n"); // Back

        // Progress Tracking -> Empty
        sb.append("3\n");
        sb.append("1\n"); // Update Task
        sb.append("\n"); // pause
        sb.append("2\n"); // Back

        // Now populate data
        // Project Setup
        sb.append("1\n");
        // Create project Invalid Name
        sb.append("1\n");
        sb.append("\n"); // Blank name -> Exception
        sb.append("\n"); // desc
        sb.append("\n"); // pause

        sb.append("1\n"); // Create project Valid
        sb.append("ProjA\n"); // name
        sb.append("DescA\n"); // desc
        sb.append("\n"); // pause

        // Project Setup -> View all (Has Project)
        sb.append("2\n");
        sb.append("\n"); // pause
        sb.append("3\n"); // Back

        // Task Assignment
        sb.append("2\n");
        // Add Task - Invalid project selection
        sb.append("1\n");
        sb.append("99\n"); // Invalid project index
        sb.append("\n"); // pause
        // Add Task - Invalid input format
        sb.append("1\n");
        sb.append("abc\n"); // Invalid index format
        sb.append("\n"); // pause
        // Add Task - Valid
        sb.append("1\n");
        sb.append("1\n"); // Select Project 1
        sb.append("TaskA\n"); // title
        sb.append("DescA\n"); // desc
        sb.append("\n"); // pause

        // Assign User - Invalid Number Format
        sb.append("2\n");
        sb.append("abc\n"); // Invalid task index format
        sb.append("1\n");
        sb.append("\n"); // pause
        // Assign User - Valid
        sb.append("2\n");
        sb.append("1\n"); // Task 1
        sb.append("1\n"); // User 1
        sb.append("\n"); // pause

        // View all Tasks
        sb.append("3\n");
        sb.append("\n"); // pause

        sb.append("4\n"); // Back

        // Progress Tracking
        sb.append("3\n");
        // Update Task - Invalid
        sb.append("1\n");
        sb.append("abc\n"); // Invalid
        sb.append("1\n");
        sb.append("\n"); // pause
        // Update Task - Valid
        sb.append("1\n");
        sb.append("1\n"); // Task 1
        sb.append("3\n"); // status 3 (DONE)
        sb.append("\n"); // pause
```

```java

        sb.append("2\n"); // Back

        // Reporting
        sb.append("4\n");
        sb.append("1\n"); // General
        sb.append("\n"); // pause
        sb.append("2\n"); // Task
        sb.append("\n"); // pause
        sb.append("3\n"); // Back

        // Logout
        sb.append("5\n");
        sb.append("\n"); // pause

        // Auth Menu -> Swap Storage Cancel
        sb.append("4\n"); // Change storage
        sb.append("4\n"); // Cancel

        // Auth Menu -> Swap Storage
        sb.append("4\n"); // Change storage
        sb.append("1\n"); // Select Binary
        sb.append("\n"); // pause

        // Exit
        sb.append("5\n");

        provideInput(sb.toString());
        try { Main.main(new String[0]); } catch(Throwable t) {}
    }
}
```

8.3 projectmanager-app/src/test/java/com/projectmanagement/lib/models/ProjectTest.java File Reference

```java
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import java.util.ArrayList;
import java.util.List;
import org.junit.jupiter.api.Assertions.*;
```

Classes

- class com.projectmanagement.lib.models.ProjectTest

Packages

- package com.projectmanagement.lib.models

8.4 ProjectTest.java

Go to the documentation of this file.

```java
package com.projectmanagement.lib.models;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

```

```java
import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

public class ProjectTest {

    private final ByteArrayOutputStream outContent = new ByteArrayOutputStream();
    private final PrintStream originalOut = System.out;

    @BeforeEach
    public void setUp() {
        System.setOut(new PrintStream(outContent));
    }

    @AfterEach
    public void tearDown() {
        System.setOut(originalOut);
    }

    @Test
    public void testDefaultConstructor() {
        Project project = new Project();
        assertNull(project.getId());
        assertNotNull(project.getCreatedDate());
        assertNotNull(project.getTasks());
        assertTrue(project.getTasks().isEmpty());
    }

    @Test
    public void testParameterizedConstructor() {
        Project project = new Project("p1", "Alpha", "Project Alpha description");
        assertEquals("p1", project.getId());
        assertEquals("Alpha", project.getName());
        assertEquals("Project Alpha description", project.getDescription());
        assertNotNull(project.getTasks());
        assertTrue(project.getTasks().isEmpty());
    }

    @Test
    public void testSettersAndGetters() {
        Project project = new Project();
        project.setId("p2");
        project.setName("Beta");
        project.setDescription("Project Beta description");

        List<Task> tasks = new ArrayList<>();
        Task task = new Task("t1", "T1", "Desc");
        tasks.add(task);
        project.setTasks(tasks);

        assertEquals("p2", project.getId());
        assertEquals("Beta", project.getName());
        assertEquals("Project Beta description", project.getDescription());
        assertEquals(1, project.getTasks().size());
        assertEquals(task, project.getTasks().get(0));
    }

    @Test
    public void testAddRemoveTask() {
        Project project = new Project();
        Task task1 = new Task("t1", "T1", "Desc1");
        Task task2 = new Task("t2", "T2", "Desc2");

        project.addTask(task1);
        project.addTask(task2);
        assertEquals(2, project.getTasks().size());
        assertTrue(project.getTasks().contains(task1));

        project.removeTask(task1);
        assertEquals(1, project.getTasks().size());
        assertFalse(project.getTasks().contains(task1));
    }

    @Test
    public void testDisplayDetails() {
        Project project = new Project("p3", "Gamma", "Proj Gamma");
        project.addTask(new Task("t1", "T1", "D1"));
        project.displayDetails();
        String output = outContent.toString();
        assertTrue(output.contains("Project ID: p3"));
        assertTrue(output.contains("Name: Gamma"));
        assertTrue(output.contains("Description: Proj Gamma"));
        assertTrue(output.contains("Total Tasks: 1"));
    }
```

```
}
```

8.5 projectmanager-app/src/test/java/com/projectmanagement/lib/↩ models/TaskTest.java File Reference

```
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import org.junit.jupiter.api.Assertions.*;
```

Classes

- class com.projectmanagement.lib.models.TaskTest

Packages

- package com.projectmanagement.lib.models

8.6 TaskTest.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.models;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.io.ByteArrayOutputStream;
import java.io.PrintStream;

import static org.junit.jupiter.api.Assertions.*;

public class TaskTest {

    private final ByteArrayOutputStream outContent = new ByteArrayOutputStream();
    private final PrintStream originalOut = System.out;

    @BeforeEach
    public void setUp() {
        System.setOut(new PrintStream(outContent));
    }

    @AfterEach
    public void tearDown() {
        System.setOut(originalOut);
    }

    @Test
    public void testDefaultConstructor() {
        Task task = new Task();
        assertNull(task.getId());
        assertNotNull(task.getCreatedDate());
        assertEquals(TaskStatus.TODO, task.getStatus());
        assertNull(task.getTitle());
        assertNull(task.getDescription());
        assertNull(task.getAssignedUser());
    }

    @Test
    public void testParameterizedConstructor() {
```

```
        Task task = new Task("t1", "Fix bug", "Fix the critical issue");
        assertEquals("t1", task.getId());
        assertEquals("Fix bug", task.getTitle());
        assertEquals("Fix the critical issue", task.getDescription());
        assertEquals(TaskStatus.TODO, task.getStatus());
        assertNull(task.getAssignedUser());
    }

    @Test
    public void testSettersAndGetters() {
        Task task = new Task();
        User user = new User("u1", "bob", "bob@example.com", "pass");
        task.setId("t2");
        task.setTitle("Develop feature");
        task.setDescription("Code the feature");
        task.setStatus(TaskStatus.IN_PROGRESS);
        task.setAssignedUser(user);

        assertEquals("t2", task.getId());
        assertEquals("Develop feature", task.getTitle());
        assertEquals("Code the feature", task.getDescription());
        assertEquals(TaskStatus.IN_PROGRESS, task.getStatus());
        assertEquals(user, task.getAssignedUser());
    }

    @Test
    public void testDisplayDetailsUnassigned() {
        Task task = new Task("t3", "Review code", "Review PR #42");
        task.displayDetails();
        String output = outContent.toString();
        assertTrue(output.contains("Task ID: t3"));
        assertTrue(output.contains("Title: Review code"));
        assertTrue(output.contains("Status: TODO"));
        assertTrue(output.contains("Assigned To: Unassigned"));
    }

    @Test
    public void testDisplayDetailsAssigned() {
        Task task = new Task("t4", "Write tests", "Increase coverage");
        task.setStatus(TaskStatus.DONE);
        User user = new User("u1", "charlie", "charlie@m.com", "pass");
        task.setAssignedUser(user);
        task.displayDetails();
        String output = outContent.toString();
        assertTrue(output.contains("Task ID: t4"));
        assertTrue(output.contains("Title: Write tests"));
        assertTrue(output.contains("Status: DONE"));
        assertTrue(output.contains("Assigned To: charlie"));
    }
}
```

8.7 projectmanager-app/src/test/java/com/projectmanagement/lib/models/UserTest.java File Reference

```
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import java.time.LocalDateTime;
import org.junit.jupiter.api.Assertions.*;
```

Classes

- class com.projectmanagement.lib.models.UserTest

Packages

- package com.projectmanagement.lib.models

8.8 UserTest.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.models;


import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import java.time.LocalDateTime;

import static org.junit.jupiter.api.Assertions.*;

public class UserTest {

    private final ByteArrayOutputStream outContent = new ByteArrayOutputStream();
    private final PrintStream originalOut = System.out;

    @BeforeEach
    public void setUp() {
        System.setOut(new PrintStream(outContent));
    }

    @AfterEach
    public void tearDown() {
        System.setOut(originalOut);
    }

    @Test
    public void testDefaultConstructor() {
        User user = new User();
        assertNull(user.getId());
        assertNotNull(user.getCreatedDate());
        assertNull(user.getUsername());
        assertNull(user.getEmail());
        assertNull(user.getPassword());
    }

    @Test
    public void testParameterizedConstructor() {
        User user = new User("u1", "johndoe", "john@example.com", "pass123");
        assertEquals("u1", user.getId());
        assertEquals("johndoe", user.getUsername());
        assertEquals("john@example.com", user.getEmail());
        assertEquals("pass123", user.getPassword());
        assertNotNull(user.getCreatedDate());
    }

    @Test
    public void testSetters() {
        User user = new User();
        user.setId("u2");
        user.setUsername("janedoe");
        user.setEmail("jane@example.com");
        user.setPassword("newpass");
        LocalDateTime now = LocalDateTime.now();
        user.setCreatedDate(now);

        assertEquals("u2", user.getId());
        assertEquals("janedoe", user.getUsername());
        assertEquals("jane@example.com", user.getEmail());
        assertEquals("newpass", user.getPassword());
        assertEquals(now, user.getCreatedDate());
    }

    @Test
    public void testDisplayDetails() {
        User user = new User("u3", "alice", "alice@example.com", "pass");
        user.displayDetails();
        String output = outContent.toString();
        assertTrue(output.contains("User ID: u3"));
        assertTrue(output.contains("Username: alice"));
        assertTrue(output.contains("Email: alice@example.com"));
    }
}
```

8.9 projectmanager-app/src/test/java/com/projectmanagement/lib/services/ProjectServiceTest.java File Reference

```
import org.junit.jupiter.api.Assertions.*;
import org.mockito.Mockito.*;
import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.storage.IRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
```

Classes

- class com.projectmanagement.lib.services.ProjectServiceTest

Packages

- package com.projectmanagement.lib.services

8.10 ProjectServiceTest.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.lib.services;
00002
00003 import static org.junit.jupiter.api.Assertions.*;
00004 import static org.mockito.Mockito.*;
00005
00006 import com.projectmanagement.lib.models.Project;
00007 import com.projectmanagement.lib.models.Task;
00008 import com.projectmanagement.lib.storage.IRepository;
00009 import org.junit.jupiter.api.BeforeEach;
00010 import org.junit.jupiter.api.Test;
00011 import org.junit.jupiter.api.extension.ExtendWith;
00012 import org.mockito.Mock;
00013 import org.mockito.junit.jupiter.MockitoExtension;
00014
00015 @ExtendWith(MockitoExtension.class)
00016 public class ProjectServiceTest {
00017
00018     @Mock
00019     private IRepository<Project> projectRepositoryMock;
00020
00021     private ProjectService projectService;
00022
00023     @BeforeEach
00024     public void setup() {
00025         projectService = new ProjectService(projectRepositoryMock);
00026     }
00027
00028     @Test
00029     public void testCreateProject_Success() {
00030         String id = "proj-1";
00031         String name = "Test Project";
00032         String desc = "Testing Project Logic";
00033
00034         projectService.createProject(id, name, desc);
00035         verify(projectRepositoryMock, times(1)).save(any(Project.class));
00036     }
00037
00038     @Test
00039     public void testCreateProject_NullInputs_ThrowsException() {
```

```
        assertThrows(IllegalArgumentException.class, () -> {
            projectService.createProject(null, "Name", "Desc");
        });

        assertThrows(IllegalArgumentException.class, () -> {
            projectService.createProject("ID", null, "Desc");
        });

        assertThrows(IllegalArgumentException.class, () -> {
            projectService.createProject("ID", "", "Desc"); // Empty string
        });

        assertThrows(IllegalArgumentException.class, () -> {
            projectService.createProject("ID", "Name", null); // null desc
        });

        verify(projectRepositoryMock, never()).save(any(Project.class));
    }

    @Test
    public void testAddTaskToProject_Success() {
        String projectId = "proj-1";
        Project mockedProject = new Project(projectId, "Test Project", "Desc");
        Task newTask = new Task("task-1", "A Task", "Doing it");

        when(projectRepositoryMock.findById(projectId)).thenReturn(mockedProject);

        projectService.addTaskToProject(projectId, newTask);

        // Project should now have 1 task
        assertEquals(1, mockedProject.getTasks().size());
        verify(projectRepositoryMock, times(1)).update(mockedProject);
    }

    @Test
    public void testAddTaskToProject_InvalidInputs_ThrowsException() {
        assertThrows(IllegalArgumentException.class, () -> {
            projectService.addTaskToProject(null, new Task());
        });

        assertThrows(IllegalArgumentException.class, () -> {
            projectService.addTaskToProject("proj-id", null);
        });

        // Test non-existent project
        when(projectRepositoryMock.findById("missing-proj")).thenReturn(null);
        assertThrows(IllegalArgumentException.class, () -> {
            projectService.addTaskToProject("missing-proj", new Task());
        });

        verify(projectRepositoryMock, never()).update(any(Project.class));
    }

    @Test
    public void testGetAllProjects() {
        java.util.List<Project> mockList = java.util.Arrays.asList(new Project("proj-1", "T", "D"));
        when(projectRepositoryMock.findAll()).thenReturn(mockList);
        java.util.List<Project> result = projectService.getAllProjects();
        assertEquals(1, result.size());
        verify(projectRepositoryMock, times(1)).findAll();
    }
}
```

8.11 projectmanager-app/src/test/java/com/projectmanagement/lib/services/ReportServiceTest.java File Reference

```
import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.User;
import com.projectmanagement.lib.models.TaskStatus;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.Arrays;
import java.util.ArrayList;
import org.mockito.Mockito.*;
```

Classes

- class com.projectmanagement.lib.services.ReportServiceTest

Packages

- package com.projectmanagement.lib.services

8.12 ReportServiceTest.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.services;

import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.User;
import com.projectmanagement.lib.models.TaskStatus;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.Arrays;
import java.util.ArrayList;

import static org.mockito.Mockito.*;

public class ReportServiceTest {

    private ProjectService mockProjectService;
    private TaskService mockTaskService;
    private UserService mockUserService;
    private ReportService reportService;

    @BeforeEach
    public void setUp() {
        mockProjectService = mock(ProjectService.class);
        mockTaskService = mock(TaskService.class);
        mockUserService = mock(UserService.class);
        reportService = new ReportService(mockProjectService, mockTaskService, mockUserService);
    }

    @Test
    public void testGenerateGeneralSystemReport() {
        // Arrange
        User u1 = new User("u1", "U1", "u1@test.com", "pass");
        User u2 = new User("u2", "U2", "u2@test.com", "pass");

        Project p1 = new Project("p1", "Project 1", "Desc");
        Task t1 = new Task("t1", "Task 1", "Desc");
        p1.addTask(t1);

        when(mockUserService.getAllUsers()).thenReturn(Arrays.asList(u1, u2));
        when(mockProjectService.getAllProjects()).thenReturn(Arrays.asList(p1));
        when(mockTaskService.getAllTasks()).thenReturn(Arrays.asList(t1));

        // Act & Assert (mostly to ensure no exceptions are thrown and it covers the lines)
        reportService.generateGeneralSystemReport();
    }

    @Test
    public void testGenerateGeneralSystemReport_Empty() {
        when(mockUserService.getAllUsers()).thenReturn(null);
        when(mockProjectService.getAllProjects()).thenReturn(null);
        when(mockTaskService.getAllTasks()).thenReturn(null);

        reportService.generateGeneralSystemReport();
    }

    @Test
    public void testGenerateGeneralSystemReport_EmptyLists() {
        when(mockUserService.getAllUsers()).thenReturn(new ArrayList<>());
        when(mockProjectService.getAllProjects()).thenReturn(new ArrayList<>());
        when(mockTaskService.getAllTasks()).thenReturn(new ArrayList<>());
        reportService.generateGeneralSystemReport();
    }

    @Test
    public void testGenerateTaskStatusReport() {
```

```
        Task t1 = new Task("t1", "T1", "Desc");
        t1.setStatus(TaskStatus.TODO);

        Task t2 = new Task("t2", "T2", "Desc");
        t2.setStatus(TaskStatus.IN_PROGRESS);

        Task t3 = new Task("t3", "T3", "Desc");
        t3.setStatus(TaskStatus.DONE);

        when(mockTaskService.getAllTasks()).thenReturn(Arrays.asList(t1, t2, t3));

        reportService.generateTaskStatusReport();
    }

    @Test
    public void testGenerateTaskStatusReport_Empty() {
        when(mockTaskService.getAllTasks()).thenReturn(new ArrayList<>());
        reportService.generateTaskStatusReport();

        when(mockTaskService.getAllTasks()).thenReturn(null);
        reportService.generateTaskStatusReport();
    }
}
```

8.13 projectmanager-app/src/test/java/com/projectmanagement/lib/↩ services/TaskServiceTest.java File Reference

```
import org.junit.jupiter.api.Assertions.*;
import org.mockito.Mockito.*;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.TaskStatus;
import com.projectmanagement.lib.models.User;
import com.projectmanagement.lib.storage.IRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
```

Classes

- class com.projectmanagement.lib.services.TaskServiceTest

Packages

- package com.projectmanagement.lib.services

8.14 TaskServiceTest.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.services;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.TaskStatus;
import com.projectmanagement.lib.models.User;
import com.projectmanagement.lib.storage.IRepository;
```

```java
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
public class TaskServiceTest {

    @Mock
    private IRepository<Task> taskRepositoryMock;

    private TaskService taskService;

    @BeforeEach
    public void setup() {
        taskService = new TaskService(taskRepositoryMock);
    }

    @Test
    public void testRegisterTask_Success() {
        String id = "task-1";
        String title = "Complete Documentation";
        String desc = "Write all the missing Doxygen comments";

        taskService.registerTask(id, title, desc);
        verify(taskRepositoryMock, times(1)).save(any(Task.class));
    }

    @Test
    public void testRegisterTask_NullFields_ThrowsException() {
        assertThrows(IllegalArgumentException.class, () -> {
            taskService.registerTask(null, "Title", "Desc");
        });

        assertThrows(IllegalArgumentException.class, () -> {
            taskService.registerTask("ID", null, "Desc");
        });

        assertThrows(IllegalArgumentException.class, () -> {
            taskService.registerTask("ID", "", "Desc"); // Empty title
        });

        assertThrows(IllegalArgumentException.class, () -> {
            taskService.registerTask("ID", "Title", null);
        });

        verify(taskRepositoryMock, never()).save(any(Task.class));
    }

    @Test
    public void testAssignTaskToUser_Success() {
        String taskId = "task-1";
        Task trackingTask = new Task(taskId, "Title", "Desc");
        User assignedUser = new User("user-1", "Bob", "bob@user.com", "hash");

        when(taskRepositoryMock.findById(taskId)).thenReturn(trackingTask);

        taskService.assignTaskToUser(taskId, assignedUser);

        assertEquals(assignedUser, trackingTask.getAssignedUser());
        verify(taskRepositoryMock, times(1)).update(trackingTask);
    }

    @Test
    public void testAssignTaskToUser_InvalidInputs_ThrowsException() {
        assertThrows(IllegalArgumentException.class, () -> {
            taskService.assignTaskToUser(null, new User());
        });

        assertThrows(IllegalArgumentException.class, () -> {
            taskService.assignTaskToUser("task-1", null);
        });

        when(taskRepositoryMock.findById("missing-task")).thenReturn(null);
        assertThrows(IllegalArgumentException.class, () -> {
            taskService.assignTaskToUser("missing-task", new User());
        });

        verify(taskRepositoryMock, never()).update(any(Task.class));
    }

    @Test
    public void testUpdateTaskStatus_Success() {
        String taskId = "task-1";
        Task operatingTask = new Task(taskId, "Title", "Desc");

```

```
        when(taskRepositoryMock.findById(taskId)).thenReturn(operatingTask);

        taskService.updateTaskStatus(taskId, TaskStatus.IN_PROGRESS);

        assertEquals(TaskStatus.IN_PROGRESS, operatingTask.getStatus());
        verify(taskRepositoryMock, times(1)).update(operatingTask);
    }

    @Test
    public void testUpdateTaskStatus_InvalidInputs_ThrowsException() {
        assertThrows(IllegalArgumentException.class, () -> {
            taskService.updateTaskStatus(null, TaskStatus.DONE);
        });

        assertThrows(IllegalArgumentException.class, () -> {
            taskService.updateTaskStatus("task-1", null);
        });

        when(taskRepositoryMock.findById("missing-task")).thenReturn(null);
        assertThrows(IllegalArgumentException.class, () -> {
             taskService.updateTaskStatus("missing-task", TaskStatus.IN_PROGRESS);
        });

        verify(taskRepositoryMock, never()).update(any(Task.class));
    }

    @Test
    public void testGetAllTasks() {
        java.util.List<Task> mockList = java.util.Arrays.asList(new Task("task-1", "T", "D"));
        when(taskRepositoryMock.findAll()).thenReturn(mockList);
        java.util.List<Task> result = taskService.getAllTasks();
        assertEquals(1, result.size());
        verify(taskRepositoryMock, times(1)).findAll();
    }
}
```

8.15 projectmanager-app/src/test/java/com/projectmanagement/lib/services/UserServiceTest.java File Reference

```
import org.junit.jupiter.api.Assertions.*;
import org.mockito.Mockito.*;
import com.projectmanagement.lib.models.User;
import com.projectmanagement.lib.storage.IRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.ArgumentCaptor;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import java.util.Arrays;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.services.UserServiceTest

Packages

- package com.projectmanagement.lib.services

8.16 UserServiceTest.java

Go to the documentation of this file.

```java
package com.projectmanagement.lib.services;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

import com.projectmanagement.lib.models.User;
import com.projectmanagement.lib.storage.IRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.ArgumentCaptor;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import java.util.Arrays;
import java.util.List;

@ExtendWith(MockitoExtension.class)
public class UserServiceTest {

    @Mock
    private IRepository<User> userRepositoryMock;

    private UserService userService;

    @BeforeEach
    public void setup() {
        userService = new UserService(userRepositoryMock);
    }

    @Test
    public void testRegisterUser_Success() {
        String id = "uuid-1";
        String username = "testuser";
        String email = "test@user.com";
        String plainPassword = "SuperSecretPassword";

        userService.registerUser(id, username, email, plainPassword);

        // Capture what was passed to create()
        ArgumentCaptor<User> userCaptor = ArgumentCaptor.forClass(User.class);
        verify(userRepositoryMock, times(1)).save(userCaptor.capture());

        User capturedUser = userCaptor.getValue();
        assertEquals(username, capturedUser.getUsername());
        assertEquals(email, capturedUser.getEmail());
        assertNotEquals(plainPassword, capturedUser.getPassword(), "Password must be hashed, not plaintext!");
    }

    @Test
    public void testRegisterUser_NullFields_ThrowsException() {
        assertThrows(IllegalArgumentException.class, () -> {
            userService.registerUser(null, "username", "email", "password");
        });

        assertThrows(IllegalArgumentException.class, () -> {
            userService.registerUser("id", null, "email", "password");
        });

        assertThrows(IllegalArgumentException.class, () -> {
            userService.registerUser("id", "username", null, "password");
        });

        assertThrows(IllegalArgumentException.class, () -> {
            userService.registerUser("id", "username", "email", ""); // Empty string
        });

        assertThrows(IllegalArgumentException.class, () -> {
            userService.registerUser("id", "username", "email", null); // null password
        });

        // Verify create was NEVER called
        verify(userRepositoryMock, never()).save(any(User.class));
    }

    @Test
    public void testLogin_Success() throws Exception {
        String username = "testuser";
        String plainPassword = "SuperSecretPassword";

        // Register to capture correctly hashed user
        userService.registerUser("uuid-1", username, "test@user.com", plainPassword);
```

```
        ArgumentCaptor<User> userCaptor = ArgumentCaptor.forClass(User.class);
        verify(userRepositoryMock, times(1)).save(userCaptor.capture());
        User registeredUser = userCaptor.getValue();

        when(userRepositoryMock.findAll()).thenReturn(Arrays.asList(registeredUser));

        User loggedInUser = userService.login(username, plainPassword);

        assertNotNull(loggedInUser);
        assertEquals(username, loggedInUser.getUsername());
    }

    @Test
    public void testLogin_InvalidCredentials() throws Exception {
        String username = "testuser";
        String plainPassword = "SuperSecretPassword";
        String wrongPassword = "WrongPassword!";

        userService.registerUser("uuid-1", username, "test@user.com", plainPassword);
        ArgumentCaptor<User> userCaptor = ArgumentCaptor.forClass(User.class);
        verify(userRepositoryMock, times(1)).save(userCaptor.capture());
        User registeredUser = userCaptor.getValue();

        when(userRepositoryMock.findAll()).thenReturn(Arrays.asList(registeredUser));

        User loggedInUser = userService.login(username, wrongPassword);
        assertNull(loggedInUser, "Should return null for wrong password.");
    }

    @Test
    public void testLogin_NullOrEmptyInputs() {
        assertNull(userService.login(null, "password"));
        assertNull(userService.login("username", null));
        assertNull(userService.login("username", ""));
    }

    @Test
    public void testLogin_NoUsersInDb() {
        when(userRepositoryMock.findAll()).thenReturn(null);
        assertNull(userService.login("user", "pass"));
    }

    @Test
    public void testGetAllUsers() {
        java.util.List<User> mockList = Arrays.asList(new User("u-1", "T", "D", "P"));
        when(userRepositoryMock.findAll()).thenReturn(mockList);
        java.util.List<User> result = userService.getAllUsers();
        assertEquals(1, result.size());
        verify(userRepositoryMock, times(1)).findAll();
    }
}
```

8.17 projectmanager-app/src/test/java/com/projectmanagement/lib/↩ storage/BinaryRepositoryTest.java File Reference

```
import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.TaskStatus;
import com.projectmanagement.lib.models.User;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.List;
import org.junit.jupiter.api.Assertions.*;
```

Classes

- class **com.projectmanagement.lib.storage.BinaryRepositoryTest**

Packages

- package com.projectmanagement.lib.storage

8.18 BinaryRepositoryTest.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.lib.storage;
00002
00003 import com.projectmanagement.lib.models.Project;
00004 import com.projectmanagement.lib.models.Task;
00005 import com.projectmanagement.lib.models.TaskStatus;
00006 import com.projectmanagement.lib.models.User;
00007 import org.junit.jupiter.api.BeforeEach;
00008 import org.junit.jupiter.api.Test;
00009 import java.util.List;
00010 import static org.junit.jupiter.api.Assertions.*;
00011
00012 class BinaryRepositoryTest {
00013
00014     private IRepository<User> userRepo;
00015     private IRepository<Project> projectRepo;
00016     private IRepository<Task> taskRepo;
00017
00018     @BeforeEach
00019     void setUp() {
00020         userRepo = new BinaryUserRepository();
00021         projectRepo = new BinaryProjectRepository();
00022         taskRepo = new BinaryTaskRepository();
00023     }
00024
00025     @Test
00026     void testUserCrud() {
00027         User user = new User("u-b1", "BName", "b@name.com", "bpass");
00028         userRepo.save(user);
00029         assertNotNull(userRepo.findById("u-b1"));
00030
00031         user.setUsername("BName2");
00032         userRepo.update(user);
00033         assertEquals("BName2", userRepo.findById("u-b1").getUsername());
00034
00035         List<User> all = userRepo.findAll();
00036         assertFalse(all.isEmpty());
00037
00038         userRepo.delete("u-b1");
00039         assertNull(userRepo.findById("u-b1"));
00040     }
00041
00042     @Test
00043     void testProjectCrud() {
00044         Project proj = new Project("p-b1", "PName", "pdesc");
00045         projectRepo.save(proj);
00046         assertNotNull(projectRepo.findById("p-b1"));
00047
00048         proj.setName("PName2");
00049         projectRepo.update(proj);
00050         assertEquals("PName2", projectRepo.findById("p-b1").getName());
00051
00052         List<Project> all = projectRepo.findAll();
00053         assertFalse(all.isEmpty());
00054
00055         projectRepo.delete("p-b1");
00056         assertNull(projectRepo.findById("p-b1"));
00057     }
00058
00059     @Test
00060     void testTaskCrud() {
00061         Task task = new Task("t-b1", "TName", "tdesc");
00062         task.setStatus(TaskStatus.TODO);
00063         taskRepo.save(task);
00064         assertNotNull(taskRepo.findById("t-b1"));
00065
00066         task.setTitle("TName2");
00067         taskRepo.update(task);
00068         assertEquals("TName2", taskRepo.findById("t-b1").getTitle());
00069
00070         List<Task> all = taskRepo.findAll();
00071         assertFalse(all.isEmpty());
00072         taskRepo.delete("t-b1");
00073         assertNull(taskRepo.findById("t-b1"));
00074     }
00075
00076     @Test
00077     void testFileExceptions() throws Exception {
00078         java.nio.file.Path userFile = java.nio.file.Paths.get("users.dat");
00079         java.nio.file.Path projFile = java.nio.file.Paths.get("projects.dat");
00080         java.nio.file.Path taskFile = java.nio.file.Paths.get("tasks.dat");
00081
00082         // Create corrupt files to trigger Exception during readObject()
```

```
        java.nio.file.Files.write(userFile, "corrupted_data".getBytes());
        java.nio.file.Files.write(projFile, "corrupted_data".getBytes());
        java.nio.file.Files.write(taskFile, "corrupted_data".getBytes());
        userRepo.findAll();
        projectRepo.findAll();
        taskRepo.findAll();

        // Delete and create directories with the same name to cause IOException on FileOutputStream
        java.nio.file.Files.deleteIfExists(userFile);
        java.nio.file.Files.createDirectory(userFile);
        java.nio.file.Files.deleteIfExists(projFile);
        java.nio.file.Files.createDirectory(projFile);
        java.nio.file.Files.deleteIfExists(taskFile);
        java.nio.file.Files.createDirectory(taskFile);

        userRepo.save(new User("id", "u", "e", "p"));
        projectRepo.save(new Project("id", "p", "d"));
        taskRepo.save(new Task("id", "t", "t"));

        // Cleanup the directories so other tests can pass
        java.nio.file.Files.deleteIfExists(userFile);
        java.nio.file.Files.deleteIfExists(projFile);
        java.nio.file.Files.deleteIfExists(taskFile);
    }
}
```

8.19 projectmanager-app/src/test/java/com/projectmanagement/lib/storage/MySQLRepositoryTest.java File Reference

```
import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.TaskStatus;
import com.projectmanagement.lib.models.User;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;
import java.sql.*;
import java.util.List;
import org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Assertions.assertNotNull;
import org.mockito.ArgumentMatchers.anyString;
import org.mockito.Mockito.when;
```

Classes

- class **com.projectmanagement.lib.storage.MySQLRepositoryTest**

Packages

- package com.projectmanagement.lib.storage

8.20 MySQLRepositoryTest.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.storage;

import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.TaskStatus;
import com.projectmanagement.lib.models.User;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

import java.sql.*;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.Mockito.when;

class MySQLRepositoryTest {

    @Mock private Connection mockConn;
    @Mock private PreparedStatement mockPstmt;
    @Mock private Statement mockStmt;
    @Mock private ResultSet mockRs;

    @Mock private PreparedStatement mockTaskPstmt;
    @Mock private ResultSet mockTaskRs;

    private AutoCloseable mocks;

    private IRepository<User> userRepo;
    private IRepository<Project> projectRepo;
    private IRepository<Task> taskRepo;

    @BeforeEach
    void setUp() throws SQLException {
        mocks = MockitoAnnotations.openMocks(this);

        when(mockConn.prepareStatement(anyString())).thenReturn(mockPstmt);
        when(mockConn.createStatement()).thenReturn(mockStmt);

        when(mockPstmt.executeQuery()).thenReturn(mockRs);
        when(mockStmt.executeQuery(anyString())).thenReturn(mockRs);

        when(mockRs.next()).thenReturn(true, true, false);
        when(mockRs.getString(anyString())).thenReturn("mockData");

        when(mockConn.prepareStatement("SELECT task_id FROM project_tasks WHERE project_id = ?")).thenReturn(mockTaskPstmt);
        when(mockTaskPstmt.executeQuery()).thenReturn(mockTaskRs);
        when(mockTaskRs.next()).thenReturn(true, false);
        when(mockTaskRs.getString("task_id")).thenReturn("mockTask1");

        userRepo = new MySQLUserRepository() {
            @Override
            protected Connection getConnection() throws SQLException {
                return mockConn;
            }
        };
        projectRepo = new MySQLProjectRepository() {
            @Override
            protected Connection getConnection() throws SQLException {
                return mockConn;
            }
        };
        taskRepo = new MySQLTaskRepository() {
            @Override
            protected Connection getConnection() throws SQLException {
                return mockConn;
            }
        };
    }

    @AfterEach
    void tearDown() throws Exception {
        mocks.close();
    }

    @Test
    void testUserCrud() throws Exception {
```

```
        User user = new User("u-m1", "MName", "m@name.com", "mpass");
        userRepo.save(user);

        User foundUser = userRepo.findById("u-m1");
        assertNotNull(foundUser);
        assertEquals("mockData", foundUser.getUsername());

        user.setUsername("MName2");
        userRepo.update(user);

        List<User> all = userRepo.findAll();
        assertEquals(1, all.size());

        userRepo.delete("u-m1");
    }

    @Test
    void testProjectCrud() throws Exception {
        Project proj = new Project("p-m1", "PName", "pdesc");
        proj.addTask(new Task("mockTask1", "TName", "TDesc"));
        projectRepo.save(proj);

        Project foundProj = projectRepo.findById("p-m1");
        assertNotNull(foundProj);
        assertEquals("mockData", foundProj.getName());

        proj.setName("PName2");
        projectRepo.update(proj);

        List<Project> all = projectRepo.findAll();
        assertEquals(1, all.size());

        projectRepo.delete("p-m1");
    }

    @Test
    void testTaskCrud() throws Exception {
        when(mockRs.getString("status")).thenReturn("TODO");

        Task task = new Task("t-m1", "TName", "tdesc");
        task.setStatus(TaskStatus.TODO);
        taskRepo.save(task);

        Task foundTask = taskRepo.findById("t-m1");
        assertNotNull(foundTask);

        task.setTitle("TName2");
        taskRepo.update(task);

        List<Task> all = taskRepo.findAll();
        assertEquals(1, all.size());

        taskRepo.delete("t-m1");
    }

    @Test
    void testSQLExceptions() throws SQLException {
        when(mockConn.prepareStatement(anyString())).thenThrow(new SQLException("Mocked Exception"));
        when(mockConn.createStatement()).thenThrow(new SQLException("Mocked Exception"));

        User user = new User("id", "name", "email", "pass");
        userRepo.save(user);
        userRepo.findById("id");
        userRepo.findAll();
        userRepo.update(user);
        userRepo.delete("id");

        Project proj = new Project("id", "name", "desc");
        projectRepo.save(proj);
        projectRepo.findById("id");
        projectRepo.findAll();
        projectRepo.update(proj);
        projectRepo.delete("id");

        Task task = new Task("id", "title", "desc");
        taskRepo.save(task);
        taskRepo.findById("id");
        taskRepo.findAll();
        taskRepo.update(task);
        taskRepo.delete("id");
    }

    @Test
    void testConstructorSQLExceptions() {
        org.junit.jupiter.api.Assertions.assertDoesNotThrow(() -> {
            new MySQLUserRepository() {
                @Override protected Connection getConnection() throws SQLException { throw new
```

```
SQLException("Mock Constructor Ex"); }
            };
            new MySQLProjectRepository() {
                @Override protected Connection getConnection() throws SQLException { throw new SQLException("Mock Constructor Ex"); }
            };
            new MySQLTaskRepository() {
                @Override protected Connection getConnection() throws SQLException { throw new SQLException("Mock Constructor Ex"); }
            };
        });
    }
}
```

8.21 projectmanager-app/src/test/java/com/projectmanagement/lib/storage/RepositoryFactoryTest.java File Reference

```
import org.junit.jupiter.api.Assertions.*;
import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.User;
import org.junit.jupiter.api.Test;
```

Classes

- class com.projectmanagement.lib.storage.RepositoryFactoryTest

Packages

- package com.projectmanagement.lib.storage

8.22 RepositoryFactoryTest.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.storage;

import static org.junit.jupiter.api.Assertions.*;

import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.User;
import org.junit.jupiter.api.Test;

public class RepositoryFactoryTest {

    @Test
    public void testGetRepository_BinaryStorage() {
        StorageConfig config = new StorageConfig(StorageType.BINARY_FILE);
        RepositoryFactory factory = new RepositoryFactory(config);

        IRepository<User> useRepo = factory.getUserRepository();
        assertTrue(useRepo instanceof BinaryUserRepository);

        IRepository<Project> projRepo = factory.getProjectRepository();
        assertTrue(projRepo instanceof BinaryProjectRepository);

        IRepository<Task> taskRepo = factory.getTaskRepository();
        assertTrue(taskRepo instanceof BinaryTaskRepository);
    }

    @Test
    public void testGetRepository_SQLiteStorage() {
```

```
        StorageConfig config = new StorageConfig(StorageType.SQLITE);
        RepositoryFactory factory = new RepositoryFactory(config);

        IRepository<User> useRepo = factory.getUserRepository();
        assertTrue(useRepo instanceof SQLiteUserRepository);

        IRepository<Project> projRepo = factory.getProjectRepository();
        assertTrue(projRepo instanceof SQLiteProjectRepository);

        IRepository<Task> taskRepo = factory.getTaskRepository();
        assertTrue(taskRepo instanceof SQLiteTaskRepository);
    }

    @Test
    public void testGetRepository_MySQLStorage() {
        StorageConfig config = new StorageConfig(StorageType.MYSQL);
        RepositoryFactory factory = new RepositoryFactory(config);

        IRepository<User> useRepo = factory.getUserRepository();
        assertTrue(useRepo instanceof MySQLUserRepository);

        IRepository<Project> projRepo = factory.getProjectRepository();
        assertTrue(projRepo instanceof MySQLProjectRepository);

        IRepository<Task> taskRepo = factory.getTaskRepository();
        assertTrue(taskRepo instanceof MySQLTaskRepository);
    }

    @Test
    public void testGetRepository_UnsupportedStorageType() {
        // Technically Java enums enforce type safety so this won't happen normally,
        // but it's good practice to ensure the switch loop handles default fallthrough errors.

        // Simulating the edge case where config gets set to null (or someone adds a new enum type but forgets to add it to the factory switch cases)
        StorageConfig config = new StorageConfig(null);
        RepositoryFactory factory = new RepositoryFactory(config);

        assertThrows(IllegalStateException.class, factory::getUserRepository);
        assertThrows(IllegalStateException.class, factory::getProjectRepository);
        assertThrows(IllegalStateException.class, factory::getTaskRepository);
    }
}
```

8.23 projectmanager-app/src/test/java/com/projectmanagement/lib/storage/SQLiteRepositoryTest.java File Reference

```
import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.TaskStatus;
import com.projectmanagement.lib.models.User;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;
import java.sql.*;
import java.util.List;
import org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Assertions.assertNotNull;
import org.mockito.ArgumentMatchers.anyString;
import org.mockito.Mockito.when;
```

Classes

- class **com.projectmanagement.lib.storage.SQLiteRepositoryTest**

Packages

- package com.projectmanagement.lib.storage

8.24 SQLiteRepositoryTest.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.storage;

import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.TaskStatus;
import com.projectmanagement.lib.models.User;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

import java.sql.*;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.Mockito.when;

class SQLiteRepositoryTest {

    @Mock private Connection mockConn;
    @Mock private PreparedStatement mockPstmt;
    @Mock private Statement mockStmt;
    @Mock private ResultSet mockRs;

    @Mock private PreparedStatement mockTaskPstmt;
    @Mock private ResultSet mockTaskRs;

    private AutoCloseable mocks;

    private IRepository<User> userRepo;
    private IRepository<Project> projectRepo;
    private IRepository<Task> taskRepo;

    @BeforeEach
    void setUp() throws SQLException {
        mocks = MockitoAnnotations.openMocks(this);

        when(mockConn.prepareStatement(anyString())).thenReturn(mockPstmt);
        when(mockConn.createStatement()).thenReturn(mockStmt);

        when(mockPstmt.executeQuery()).thenReturn(mockRs);
        when(mockStmt.executeQuery(anyString())).thenReturn(mockRs);

        when(mockRs.next()).thenReturn(true, true, false);
        when(mockRs.getString(anyString())).thenReturn("mockData");

        when(mockConn.prepareStatement("SELECT task_id FROM project_tasks WHERE project_id = ?")).thenReturn(mockTaskPstmt);
        when(mockTaskPstmt.executeQuery()).thenReturn(mockTaskRs);
        when(mockTaskRs.next()).thenReturn(true, false);
        when(mockTaskRs.getString("task_id")).thenReturn("mockTask1");

        userRepo = new SQLiteUserRepository() {
            @Override
            protected Connection getConnection() throws SQLException {
                return mockConn;
            }
        };
        projectRepo = new SQLiteProjectRepository() {
            @Override
            protected Connection getConnection() throws SQLException {
                return mockConn;
            }
        };
        taskRepo = new SQLiteTaskRepository() {
            @Override
            protected Connection getConnection() throws SQLException {
                return mockConn;
            }
        };
```

```
    }

    @AfterEach
    void tearDown() throws Exception {
        mocks.close();
    }

    @Test
    void testUserCrud() throws Exception {
        User user = new User("u-s1", "SName", "s@name.com", "spass");
        userRepo.save(user);

        User foundUser = userRepo.findById("u-s1");
        assertNotNull(foundUser);
        assertEquals("mockData", foundUser.getUsername());

        user.setUsername("SName2");
        userRepo.update(user);

        List<User> all = userRepo.findAll();
        assertEquals(1, all.size());

        userRepo.delete("u-s1");
    }

    @Test
    void testProjectCrud() throws Exception {
        Project proj = new Project("p-s1", "PName", "pdesc");
        proj.addTask(new Task("mockTask1", "Title", "Desc"));
        projectRepo.save(proj);

        Project foundProj = projectRepo.findById("p-s1");
        assertNotNull(foundProj);
        assertEquals("mockData", foundProj.getName());

        proj.setName("PName2");
        projectRepo.update(proj);

        List<Project> all = projectRepo.findAll();
        assertEquals(1, all.size());

        projectRepo.delete("p-s1");
    }

    @Test
    void testTaskCrud() throws Exception {
        when(mockRs.getString("status")).thenReturn("TODO");

        Task task = new Task("t-s1", "TName", "tdesc");
        task.setStatus(TaskStatus.TODO);
        taskRepo.save(task);

        Task foundTask = taskRepo.findById("t-s1");
        assertNotNull(foundTask);

        task.setTitle("TName2");
        taskRepo.update(task);

        List<Task> all = taskRepo.findAll();
        assertEquals(1, all.size());

        taskRepo.delete("t-s1");
    }

    @Test
    void testSQLExceptions() throws SQLException {
        when(mockConn.prepareStatement(anyString())).thenThrow(new SQLException("Mocked Exception"));
        when(mockConn.createStatement()).thenThrow(new SQLException("Mocked Exception"));

        User user = new User("id", "name", "email", "pass");
        userRepo.save(user);
        userRepo.findById("id");
        userRepo.findAll();
        userRepo.update(user);
        userRepo.delete("id");

        Project proj = new Project("id", "name", "desc");
        projectRepo.save(proj);
        projectRepo.findById("id");
        projectRepo.findAll();
        projectRepo.update(proj);
        projectRepo.delete("id");

        Task task = new Task("id", "title", "desc");
        taskRepo.save(task);
        taskRepo.findById("id");
        taskRepo.findAll();
```

```
        taskRepo.update(task);
        taskRepo.delete("id");
    }

    @Test
    void testConstructorSQLExceptions() {
        org.junit.jupiter.api.Assertions.assertDoesNotThrow(() -> {
            new SQLiteUserRepository() {
                @Override protected Connection getConnection() throws SQLException { throw new SQLException("Mock Constructor Ex"); }
            };
            new SQLiteProjectRepository() {
                @Override protected Connection getConnection() throws SQLException { throw new SQLException("Mock Constructor Ex"); }
            };
            new SQLiteTaskRepository() {
                @Override protected Connection getConnection() throws SQLException { throw new SQLException("Mock Constructor Ex"); }
            };
        });
    }
}
```